

**BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE
SPECIALIZATION ARTIFICIAL INTELLIGENCE**

DIPLOMA THESIS

3D Object Reassembly of Broken Objects using Deep Learning Methods

Supervisor

**Dr. ANDREICA Anca, University Professor
CHIŞ Raluca-Diana, PhD candidate**

Author
GHEORGHE Andreea-Ioana

2026

Declaration on the Use of Generative AI Tools

The following generative artificial intelligence tools were used during the preparation of this thesis: **Claude** (Anthropic) and **ChatGPT** (OpenAI).

These tools were used for the following purposes:

- **Reformulating and improving academic text:** generative AI tools were used as a starting point for reformulating ideas and improving the readability of the English text. All generated formulations were reviewed, adapted, and integrated by the author, preserving the original content and contributions of this work.
- **Identifying and resolving code errors:** the tools were used to identify bugs in the source code implementation, suggest debugging solutions, and verify the correctness of code fragments.

Generative AI tools were **not** used for:

- generating original content (research ideas, contributions, experimental analysis);
- writing complete sections of the thesis without personal review and reformulation;
- generating or falsifying experimental results.

The author assumes full responsibility for the content of this thesis and for the results obtained. The use of generative AI tools was carried out with discernment and integrity, in accordance with the university regulations.

Abstract

Reconstructing a complete 3D object from its fractured pieces is a long-standing problem with applications in archaeology, paleontology, medicine, and forensics. Given point clouds of the broken fragments, the task is to predict the rigid transformation mapping each piece to its original position, a 6-Degree-of-Freedom pose estimation problem. This thesis investigates whether targeted modifications to the Jigsaw pipeline, the first end-to-end learning-based framework for multi-piece fracture assembly, can further improve reconstruction quality, and what such experiments reveal about the relative importance of architecture, data, and optimization.

Four extensions are proposed and evaluated: (1) a geometric pair attention bias adapted from the GPAT architecture, which acts as a learned soft regularizer that smooths cross-attention distributions; (2) a gated double attention module that adds a second round of self- and cross-attention through some learned gates; (3) a Gabriel graph neighborhood filter that replaces standard k -nearest neighbors with surface-faithful connectivity in the PointNet++ backbone; and (4) an Iterative Closest Point (ICP) refinement step inserted between RANSAC-based pose estimation and global synchronization. All experiments are conducted on the everyday and artifact subsets of Breaking Bad, with models finetuned on both two-piece and multi-piece (2-4) fracture patterns.

The pair attention regularization consistently improves out-of-distribution generalization on the artifact subset while remaining neutral on in-distribution data. Gated double attention provides +2 to 5 percentage points under data scarcity and consistent cross-domain gains on the artifact subset. Gabriel graph filtering improves cross-domain pose quality on artifact objects, producing higher-quality matches, though the reduced neighborhood diversity slightly lowers the overall correspondence count. ICP refinement adds approximately 1 percentage point across all configurations as a reliable, non-learned enhancement. A data scaling analysis comparing the full dataset against a reduced subset ($\sim 63\%$) reveals that data quantity has a large compounding effect, with the full dataset yielding over 10 additional percentage points. An interactive web application accompanies the thesis, allowing users to run inference across eleven model variants and visually compare reconstruction quality in 3D.

Contents

Declaration on the Use of Generative AI Tools	i
Abstract	ii
1 Introduction	1
1.1 Objectives	3
1.2 Original Contributions	3
1.3 Thesis Structure	4
2 Problem definition	6
2.1 Taxonomy of 3D Assembly Problems	6
2.2 Problem Decomposition	8
3 Theoretical Background	9
3.1 Point Cloud Representation	9
3.2 PointNet and PointNet++	10
3.3 Transformers and Attention Mechanisms	12
3.4 RANSAC	13
4 Related Work	15
4.1 Historical Evolution of 3D Shape Reassembly Methodologies	15
4.2 New generation of SOTA	17
4.2.1 The Efficiency Bottleneck	17
4.2.2 The Attention Bottleneck	19
4.2.3 The Blindness Bottleneck	21
4.2.4 The Synthetic-to-Real Gap	22
4.2.5 Summary and Research Gap	24
5 Methodology: Proposed Extensions to Jigsaw	26
5.1 Baseline Pipeline and Modification Points	26
5.2 Front-End Feature Extractor	27

5.2.1	Proposed Modification: Pair Geometric Attention as Soft Regularization	28
5.2.2	Proposed Improvement: Gated Double Attention	30
5.2.3	Proposed Extension: Gabriel Graph Neighborhood Filtering	31
5.3	Fracture Surface Segmentation	33
5.4	Multi-Part Point Matching	34
5.5	Pose Recovery and Alignment	35
5.5.1	Proposed Improvement: ICP Refinement for Pairwise Poses	36
6	Experiments and Results	39
6.1	Experimental Setup	39
6.1.1	The Breaking Bad Dataset	39
6.1.2	Evaluation Metrics	40
6.1.3	Implementation Details	41
6.1.4	Scope and Limitations	44
6.2	Context: Published Results on Breaking Bad	44
6.3	Results on the Full Dataset	45
6.3.1	Two-Piece Assembly	45
6.3.2	Multi-Piece Assembly (2 to 4 Pieces)	46
6.4	Data Scaling: Training on a Reduced Subset	46
6.5	The Effect of Loss Scheduling	48
6.6	Gate Behavior Analysis	49
6.7	Gabriel Graph: Retention Ratio Analysis	50
6.8	Discussion	51
7	System Implementation and Application	53
7.1	System Overview	53
7.2	Architecture	53
7.3	Key Implementation Choices	55
7.3.1	User Workflow	56
7.3.2	Functional Requirements	58
7.4	Software Quality Considerations	58
7.5	Testing	59
8	Conclusions, Threats to Validity, and Future Work	61
8.1	Conclusions	61
8.2	Threats to Validity	63
8.3	Future Work	63
	Bibliography	66

A	Application Details	70
A.1	Data Organization	70
A.2	API Endpoints	70
A.3	Non-Functional Requirements	71

Chapter 1

Introduction

The ability to look at a collection of broken fragments and reason about how they once fit together is a form of spatial intelligence that humans have practiced for millennia. In archaeology, researchers reassemble eroded pottery, shattered frescoes, and fragmented statues to preserve the material record of ancient civilizations [HFG⁺06, TFBW⁺10, FSTF⁺11]. In paleontology, fractured bones and lithic artifacts are pieced together to reconstruct tools and skeletal remains. In medicine, surgeons mentally reconstruct comminuted bone fractures before planning an operation.

Despite the importance of these tasks, they remain overwhelmingly manual. An experimental study on lithic refitting reported a success rate of only 30%, with experts performing only marginally better than novices [LK10]. Museum storerooms around the world hold thousands of unassembled fragments, waiting to be pieced together by specialists whose time is limited and whose accuracy, even with expertise, is modest. The sheer scale of the problem, combined with the risk of further damaging fragile artifacts through physical handling, has long motivated the search for computational alternatives.

With the advent of high-resolution 3D scanning (Figure 1.1), the physical pieces of a broken object can now be digitized into point clouds: unordered sets of 3D coordinates that capture the geometry of each fragment. Given a set of such point clouds, the goal is to predict the rigid transformation (a rotation and a translation) that maps each fragment back to its original position. The problem is known as 6-Degree-of-Freedom (6-DoF) pose estimation and sits at the intersection of computer vision, computational geometry, and deep learning.

Automating 3D reassembly is far from trivial. Unlike 2D jigsaw puzzles, where flat edges and printed patterns constrain the solution, 3D fracture fragments feature irregular, continuous surfaces with no discrete boundary markers. The search space grows combinatorially with the number of pieces, and small errors in local pairwise alignments propagate and accumulate when assembling multi-part objects, a phenomenon known as drift.

Early computational approaches relied on hand-crafted geometric descriptors (integral invariants [HFG⁺06], curvature profiles, surface normals, and boundary contours [TFBW⁺10]) to find matches between fracture surfaces. While effective on clean, controlled breaks, these methods are sensitive to surface ambiguity and deteriorate when applied to eroded, incomplete, or noisy real-world fragments. A parallel line of research addressed semantic part assembly, reconstructing objects from predefined, functionally meaningful components such as furniture legs or chair backs [HZF⁺20, HNR22, WZX⁺20]. These methods leverage categorical priors and part labels, but their assumptions break down when applied to arbitrary fracture fragments that carry no semantic identity.

The landscape shifted with two key developments. First, specialized neural architectures for unstructured 3D data, PointNet [QSMG17] and PointNet++ [QYSG17], enabled the direct processing of point clouds, replacing hand-crafted features with learned geometric representations. Second, large-scale fracture datasets, most notably Breaking Bad [S⁺22], provided the training data necessary for data-driven approaches to geometric assembly. Building on these foundations, the Jigsaw framework [LSH23] established the first learning-based pipeline for multi-piece fracture assembly, combining hierarchical point cloud features with attention mechanisms [VSP⁺17], global bipartite matching via Sinkhorn optimal transport [Cut13], and transformation synchronization through Shonan averaging [DRW⁺20]. Jigsaw set the standard baseline for subsequent research on Breaking Bad.

Since Jigsaw, the field has evolved rapidly along several directions. To address the computational cost of dense point matching, PMTR [LML⁺24] introduced sub-quadratic proxy tensors, while PHFormer [CYD24] decomposed pose estimation into adjacency prediction and transformation regression using proxy-level attention. To enrich attention with spatial reasoning, the Geometric Point Attention Transformer (GPAT) [LCML23] demonstrated that encoding pairwise distances and angles between fragments directly into the attention scores can improve assembly quality. To introduce global shape awareness, Jigsaw++ [LHH24] incorporated a shape prior through latent space reconstruction using Rectified Flow. To bridge the gap between synthetic training data and real-world fractures, GARF [LJC⁺25] recast the problem as learning continuous velocity fields on the Lie group $SE(3)$, achieving state-of-the-art performance by moving fragments along geodesic paths.

Despite this rapid progress, several open questions remain within the Jigsaw pipeline, which continues to serve as the foundational baseline for fracture assembly on Breaking Bad. Its cross-attention mechanism can produce overconfident correspondence distributions, particularly on out-of-distribution objects, and may benefit from regularization techniques that smooth the attention weights. The feature extractor uses a single round of attention, while recent work (PHFormer, GARF)

suggests that iterative refinement of self-attention and cross-attention improves geometric representations. The PointNet++ backbone constructs local neighborhoods via k -nearest neighbors without regard for surface topology, potentially including spatially close but topologically distant points in each centroid’s neighborhood. And pairwise pose estimates are recovered from sparse RANSAC correspondences without fully exploiting the dense geometry available on fracture surfaces.



(a) Digital reconstruction of a primate statue: raw 3D scan, point cloud, and textured mesh.

(b) High-resolution structured-light 3D scanning of a cultural heritage artifact.

Figure 1.1: The digitization pipeline for cultural heritage preservation. (a) A fractured artifact is scanned to produce point clouds and 3D meshes suitable for computational reassembly. (b) The physical scanning process using structured-light technology.

1.1 Objectives

The objective of this thesis is to investigate whether targeted, modular modifications to the Jigsaw framework can address the limitations outlined above, and what such experiments reveal about the relationship between architecture, data, and optimization in 3D fracture assembly. Rather than proposing a new architecture, we study four specific extensions: a geometric attention bias for cross-attention, a gated second attention pass, a Gabriel graph neighborhood filter for PointNet++, and an ICP refinement step for pose recovery; alongside a data scaling analysis and an interactive web application for visual comparison of results.

1.2 Original Contributions

The main contributions of this thesis are:

1. **Geometric pair attention as learned soft regularization.** We integrate the pair attention mechanism from GPAT into Jigsaw’s cross-attention layers, where it learns a data-dependent additive bias that softens the attention distribution. This acts as a learned regularizer that consistently improves out-of-distribution

generalization on the artifact subset while remaining neutral on in-distribution performance, a pattern that holds across data regimes and piece counts.

2. **Gated double attention with training dynamics analysis.** We extend the feature extractor with a second attention pass using zero-initialized ReZero gates. On a reduced dataset the additional capacity acts as an implicit regularizer, providing +2 to 5 percentage points. On the full dataset, the baseline already captures these patterns, confirming that data and architecture address the same representational bottleneck from complementary directions. We further analyze the learned gate values and demonstrate the sensitivity of gated finetuning to loss scheduling.
3. **Gabriel graph neighborhood filtering.** We replace the standard kNN grouping in PointNet++ with a Gabriel graph filter that prunes topologically distant neighbors from each centroid’s neighborhood. The experiment shows that surface-faithful neighborhoods produce higher-quality matches and improve cross-domain pose metrics on artifact objects, but the reduced neighborhood diversity slightly lowers the overall correspondence count. A systematic study of the retention ratio r reveals the trade-off between pruning aggressiveness and feature aggregation quality.
4. **ICP refinement for pairwise pose estimation.** Point-to-point ICP inserted between RANSAC and Shonan averaging yields a consistent improvement of approximately 1 percentage point across all configurations. The modification is non-learned, orthogonal to the attention-based extensions, and carries no risk of overfitting.
5. **Data scaling analysis.** By comparing results on the full dataset against a reduced subset ($\sim 63\%$ of everyday training patterns), we quantify the effect of data quantity on assembly quality (the full dataset yields over 10 percentage points more) and show that the proposed architectural modifications provide consistent improvements under data scarcity, acting as implicit regularizers that complement the baseline.
6. **Interactive web application.** A Django and FastAPI-based demo with Three.js 3D visualization, enabling users to run inference across eleven model variants and visually compare reconstruction quality.

1.3 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2** defines the fracture assembly problem.
- **Chapter 3** covers the theoretical foundations (point cloud processing, attention mechanisms, pose estimation).
- **Chapter 4** reviews related work.
- **Chapter 5** presents the proposed extensions to the Jigsaw pipeline.
- **Chapter 6** reports the experimental evaluation.
- **Chapter 7** describes the interactive web application.
- **Chapter 8** concludes with findings, threats to validity, and future work.

Chapter 2

Problem definition

In this chapter, we formally define the 3D fractured object reassembly problem and position it within the broader context of 3D shape assembly. We begin by distinguishing between semantic part assembly and geometric fracture assembly, highlighting the fundamental differences in assumptions and challenges. Next, we introduce the mathematical formulation of the problem as a 6-DoF pose estimation task over point cloud representations. Finally, we outline the scope and objectives of this work by decomposing the problem into pairwise matching and global assembly, motivating our focus on the pairwise setting as a means to better understand and address the core challenges of local geometric alignment.

2.1 Taxonomy of 3D Assembly Problems

The field of 3D shape assembly is broadly divided into two primary sub-tasks: semantic part assembly, which deals with predefined, meaningful components, and geometric fracture assembly, which puts together irregular broken fragments.

Part assembly refers to the process of assembling components that are decomposed in a semantically meaningful way, such as furniture components (e.g., table legs) or mechanical parts (such as screws). Although closely related to the fracture assembly problem, part assembly introduces several key differences. It generally assumes that each part has an independent semantic function and that each component is complete and functional on its own, unlike broken pieces from fracture assembly which are incomplete and lack semantic meaning (Figure 2.1).

While fracture assembly relies primarily on local and global geometric cues, part assembly depends more on semantic and functional understanding of individual components and their roles within the final object. Both paradigms implicitly incorporate prior knowledge: semantic priors in part assembly and geometric priors in fracture assembly, including global shape consistency and pairwise fragment com-

patibility. Many existing approaches leverage semantic segmentation to guide the assembly process; however, this assumption becomes limiting in geometric assembly scenarios, where such labels are unavailable and reconstruction must rely solely on geometric information.

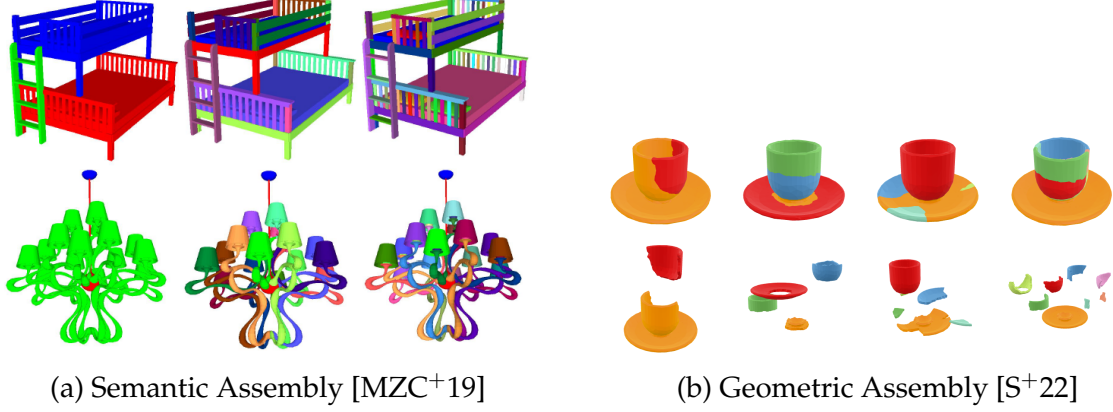


Figure 2.1: Comparison of 3D assembly paradigms. (a) Semantic part assembly operates on predefined, functionally meaningful components such as furniture parts [MZC⁺19]. (b) Geometric fracture assembly deals with irregular, physically broken fragments that lack semantic identity [S⁺22].

The reassembly of 3D fractured objects is formulated as a 6-DoF (Degree-of-Freedom) pose regression problem. To solve this, the fractured pieces are typically represented as 3D point clouds, $P = \{P_1, P_2, \dots, P_n\}$. The objective of the 3D reassembly problem becomes predicting a set of continuous rigid transformations $T = \{(R_i, t_i)\}_{i=1}^N$ where $t_i \in \mathbb{R}^3$ is a 3D translation vector and $R_i \in SO(3)$ represents a 3D rotation matrix (or quaternion). These transformations map each fragment to its original canonical position.

The canonical coordinate system is defined relative to the largest fragment, which serves as a fixed reference anchor. By applying these estimated transformations, the fragments are spatially aligned in a shared coordinate system, giving the reconstruction of the complete undeformed object:

$$O = T_1(P_1) \cup T_2(P_2) \cup \dots \cup T_n(P_n) \quad (2.1)$$

For simplicity, we assume that all fragments are available and that no extraneous pieces are present. Under these assumptions, reconstruction quality is evaluated using metrics such as mean absolute error (MAE) and root mean square error (RMSE) for both rotation and translation, as well as part accuracy (PA), defined based on a Chamfer distance threshold.

2.2 Problem Decomposition

In the field of 3D fractured object reassembly, this problem is typically modeled in two stages: *pairwise matching* and *global assembly*.

Pairwise Matching (Local Alignment): This stage focuses on evaluating the geometric compatibility between two fragments to estimate their relative rigid transformation in $SE(3)$ space. The objective is to maximize local consistency, ensuring that the micro-geometry of the fracture surfaces locks together precisely.

Global Assembly (Multi-Piece Alignment): When dealing with more than two pieces, relying on sequential pairwise matches creates a major vulnerability: small inaccuracies in local alignments progressively transfer to the next pieces, leading to error accumulation (drift). Global assembly addresses this issue by jointly optimizing the poses of all fragments simultaneously. It utilizes global structural constraints, such as cycle-consistency, to distribute alignment errors evenly across all pieces and eliminate incorrect matches.

While global assembly is essential for multi-part objects, its performance depends heavily on the accuracy of the underlying pairwise matches. Therefore, the core analysis in this work focuses on two-piece assembly, isolating the pairwise alignment stage from the error-correcting effect of global optimization. This setting targets the fundamental challenge of learning robust local geometric features and attention mechanisms. To validate that the findings generalize beyond the pairwise case, the experimental evaluation additionally includes multi-piece fracture patterns (2 to 4 fragments), where the full Jigsaw pipeline, including Shonan averaging for global synchronization, is exercised.

Chapter 3

Theoretical Background

This chapter presents the theoretical foundations underlying the proposed approach: point cloud representations and their structural properties, deep learning architectures for point sets (PointNet, PointNet++), transformer-based attention mechanisms, and the RANSAC algorithm for robust rigid transformation estimation.

3.1 Point Cloud Representation

A common representation for 3D geometry in learning-based pipelines is the point cloud, usually chosen due to their simplicity and flexibility. A point cloud is an unstructured set of 3D points, where each point p_i may include additional attributes such as color or surface normals. Unlike volumetric voxel grids, which suffer from cubic memory costs and resolution limits, or 3D meshes, which introduce complex and irregular topological connectivity, point clouds provide a highly efficient raw geometric representation. However, effectively processing these raw spatial data requires neural network architectures to explicitly account for three intrinsic structural properties:

- **Permutation invariance;** Because point clouds lack a rigid spatial grid, the physical shape they represent remains identical regardless of how the points are ordered in memory. Consequently, any model processing them must be permutation invariant, ensuring it produces the exact same output regardless of the input data sequence.
- **Local geometric structure;** A single isolated point provides no meaningful contextual information. Identifiable geometric features, such as fracture edges, corners, or smooth surfaces, emerge only from the spatial proximity and interaction between neighboring points. Models must therefore be capable of grouping points to capture these localized structures.

- **Rigid transformation invariance;** The intrinsic geometry of an object does not change based on its location or orientation. Therefore, the learned representations must remain consistent under global rigid transformations, such as 3D translations and rotations in space.

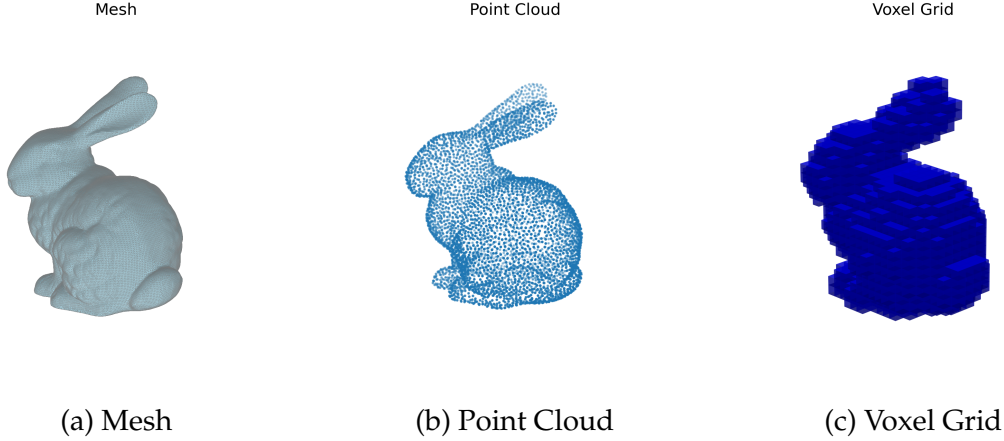


Figure 3.1: Common 3D shape representations. (a) A triangular mesh encodes surface topology through vertices and faces. (b) A point cloud represents geometry as an unordered set of 3D coordinates. (c) A voxel grid discretizes 3D space into a regular volumetric lattice.

Because of these unique properties, standard Convolutional Neural Networks are fundamentally ill-equipped to process point clouds directly (Figure 3.1). Classical convolutions are strictly based on the fixed spatial order and uniform density of regular grids. Since point clouds are unordered sets scattered continuously in 3D space, traditional convolutional operations cannot be applied, necessitating the use of specialized, order-agnostic architectures.

3.2 PointNet and PointNet++

To address the inherent challenges of processing unstructured 3D data, early research introduced architectures designed specifically for point sets, like PointNet [QSMG17]. Instead of aggregating information from local neighbors using a spatial kernel and extracting a feature from a local patch of pixels, PointNet learns features for each point independently. It replaces the convolutional kernel with a shared Multi-Layer Perceptron (MLP), denoted $h : \mathbf{R}^3 \rightarrow \mathbf{R}^K$, which acts as a pointwise feature extractor. For every individual point x_i , the network computes a high-dimensional feature vector $h(x_i)$.

Because these features are extracted independently, a permutation-invariant aggregation is needed to capture global structure. PointNet achieves this through

global max pooling:

$$g = \max_{i=1 \dots n} h(x_i) \quad (3.1)$$

Because $\max$ is a symmetric function, the output is invariant to input ordering. While CNNs build global understanding by progressively combining localized structures, PointNet aggregates the most dominant pointwise responses directly (Figure 3.2).

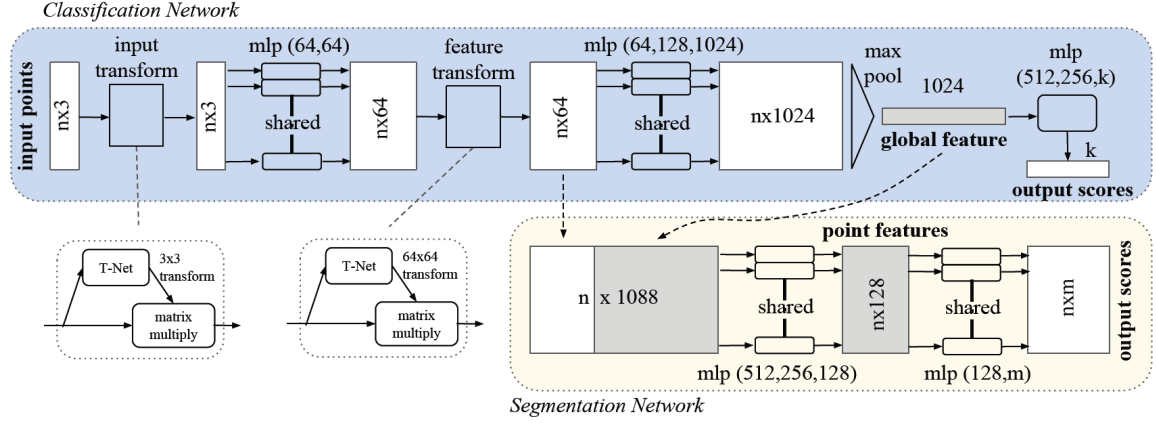


Figure 3.2: The PointNet architecture [QSMG17]. Each point is independently transformed by a shared MLP, and a symmetric max-pooling operation aggregates the per-point features into a single global descriptor.

While the foundational PointNet formulation successfully guarantees permutation invariance through global max-pooling, it introduces a critical limitation: by design, it does not capture the local structures such as spatial distances and geometric interactions. To overcome this lack of local shape awareness, PointNet++ [QYSG17] introduces a hierarchical architecture that applies PointNet recursively on a nested partitioning of the input point set. By exploiting the underlying metric space, this hierarchy allows the network to capture fine-grained local features at continuously increasing contextual scales, analogous to the progressively larger receptive fields found in CNNs.

This hierarchical feature learning is achieved through an encoder made of repeated Set Abstraction levels. Each Set Abstraction level progressively downsamples the point cloud by using Farthest Point Sampling [QYSG17] to select a sparse set of centroids, groups neighboring points within a fixed metric radius, and applies a miniature PointNet to encode these localized regions into highly descriptive feature vectors. Because real-world point clouds often suffer from non-uniform sampling densities, PointNet++ also employs density-adaptive strategies, such as Multi-Scale Grouping, which intelligently combine features extracted from neighborhoods of varying sizes to ensure robustness (Figure 3.3).

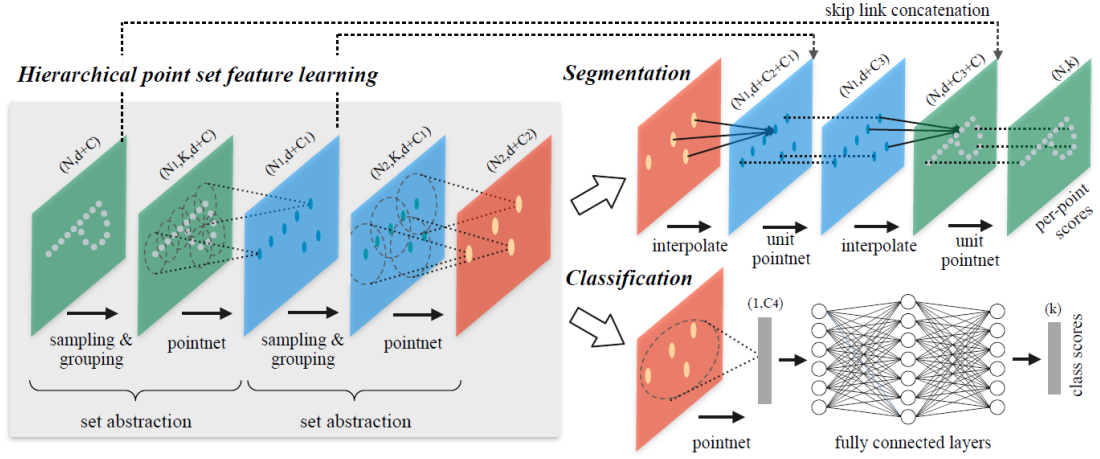


Figure 3.3: The PointNet++ architecture [QYSG17]. Hierarchical Set Abstraction levels progressively downsample the point cloud, applying local PointNet modules at each scale to capture fine-grained geometric features with increasing contextual scope.

3.3 Transformers and Attention Mechanisms

Although PointNet++ captures local geometric structures through hierarchical neighborhoods, max-pooling discards the relationships between points within each region. For tasks such as fracture boundary matching, the network must understand how different points relate to one another, not just which features are most prominent. Modern architectures address this by replacing max-pooling with the Attention mechanism [VSP⁺17], the core component of Transformer networks.

Originally designed for natural language processing, transformers have proven effective for 3D point cloud processing. The self-attention operator at their core is a set operator: it computes relationships by comparing elements against one another rather than relying on a fixed grid or sequential order, making it invariant to both the permutation and cardinality of the input. This makes the transformer well suited for processing unordered point clouds.

At its core, the attention mechanism maps a Query and a set of Key-Value pairs to an output representation. Each point’s features are linearly projected into a Query vector (encoding what the point is searching for), a Key vector (encoding its structural identity), and a Value vector (carrying its feature content). The network computes dot products between Queries and Keys to measure pairwise compatibility; these scores are normalized via softmax to produce attention weights that determine how much each point’s Value contributes to the output. Unlike the hard selection of max pooling, attention computes a dynamically weighted sum based on structural context.

In the context of 3D fractured object reassembly, architectures like Jigsaw [LSH23] utilize two distinct variations of this mechanism to establish geometric matches:

Self-Attention. In this layer, the Queries, Keys, and Values all come from the same point cloud. The network relates different positions within a single fractured piece to build a highly refined understanding of its own internal geometry and structural context.

Cross-Attention. Once the internal geometry is understood, the network must find how two different pieces fit together. In cross-attention, the Queries come from the feature representations of Fragment A, while the Keys and Values come from Fragment B. This allows the points on one fractured surface to explicitly search the geometry of the other piece, facilitating the communication of local features across different fragments to find the exact point-to-point correspondences required for assembly.

3.4 RANSAC

The Random Sample Consensus (RANSAC) [FB81] algorithm is a robust mathematical estimation paradigm introduced by Fischler and Bolles. Rather than utilizing the entire dataset to compute an initial model and subsequently filtering out errors, RANSAC takes the opposite approach: it uses the smallest feasible subset of data to instantiate a model hypothesis, and then evaluates how much of the remaining data consistently aligns with that model.

Formally, given a dataset P and a model requiring n points to instantiate, RANSAC randomly selects a subset of n points to generate a hypothesis model, M_1 . This model is then tested against all remaining points in P . Points that fall within a predefined error tolerance of M_1 are classified as "inliers" and form the consensus set. If the size of this consensus set exceeds a specific threshold t , the model is accepted, and a refined model is typically recomputed using all identified inliers. If the consensus set is too small, the algorithm randomly samples a new subset of n points and repeats the process. After a predetermined number of iterations, the transformation that produced the largest consensus set is selected as the final solution.

The Jigsaw framework adopts this exact methodology to filter out false point-to-point correspondences predicted between fracture surfaces. Because neural networks inevitably predict some erroneous matches (outliers) that would severely degrade a global pose estimation, RANSAC is used to isolate the reliable geometric matches. In $SE(3)$ space, the minimum number of points required to determine a rigid transformation is 3. Therefore, RANSAC randomly samples 3 corresponding point pairs from the two fragments.

To compute the transformation hypothesis from these 3 pairs, the algorithm solves the orthogonal Procrustes problem via Singular Value Decomposition (SVD) [GVL13]: the points are centered, the cross-covariance matrix is decomposed, and the optimal rotation is extracted while the translation is recovered from the centroid offset. The full SVD-based derivation is presented in Chapter 5 (Equation 5.15), where it is also used for differentiable pose estimation during training.

Once this candidate transformation is calculated, it is applied to all other matched points to determine the consensus set. By repeating this hypothesis-and-evaluation loop, RANSAC successfully discards the hallucinated matches and outputs a highly reliable pairwise transformation based purely on the geometric inliers.

Chapter 4

Related Work

In the following chapter, we review the evolution of methodologies in the literature designed to solve the 3D reassembly problem.

4.1 Historical Evolution of 3D Shape Reassembly Methodologies

Before the advent of deep learning, the field relied on **Traditional Geometry-Based Fracture Assembly**. These early methods utilized hand-crafted mathematical invariants and surface characteristics to align broken fragments. For example, Huang et al. [HFG⁺06] proposed reassembling fractured objects via geometric matching using integral invariants to analyze fracture surfaces. In archaeological contexts, Toler et al. [TFBW⁺10]. utilized multi-feature matching, combining color, normal maps, and geometric features to reassemble fresco fragments, while Funkhouser et al. [FSTF⁺11] applied machine learning classifiers to evaluate these hand-crafted features and learn how to match the fragments effectively (Figure 4.1).

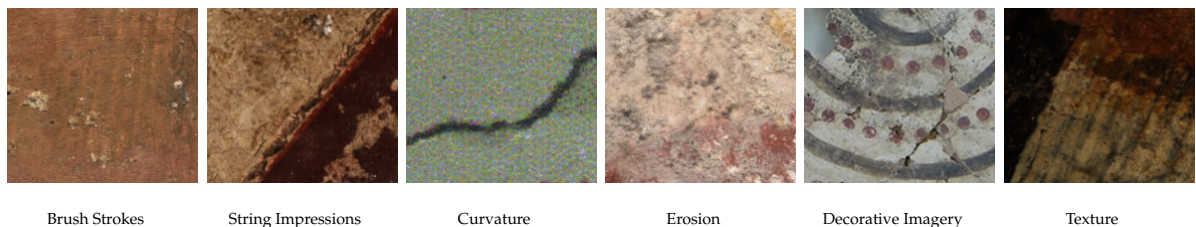


Figure 4.1: Examples of hand-crafted surface feature descriptors used in traditional fragment matching methods [TFBW⁺10]. Each descriptor captures a different physical property of the fragment surface, from brush stroke patterns to curvature profiles.

Because pure geometric matching struggles when fragments are severely damaged or missing, a sub-branch of research focused on **Template-Guided Assembly**. These methods rely on a complete reference model to guide the placement of fragments. A prominent example is the automatic assembly and completion framework proposed by Yin et al. [YWLM11], which roughly aligns fragmented skulls using a healthy template skull before refining the break-curve matches (Figure 4.2).

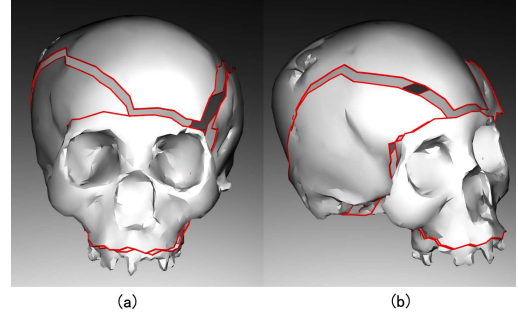


Figure 4.2: Template-guided reassembly of a fractured skull [YWLM11].

With the rise of neural networks, the focus shifted towards **Deep Learning for Semantic Part Assembly**. Rather than physical fractures, these models focused on constructing complete objects from predefined, semantically meaningful parts (e.g., table legs or chair backs). Pioneering methods in this domain include Generative 3D Part Assembly via Dynamic Graph Learning (DGL), which utilizes iterative graph message passing to reason about spatial relationships and refine part poses. [HZF⁺20]. This was subsequently improved by RGL-Net, a recurrent graph learning framework [HNR22]. Other approaches used different constraints, such as predicting 3D part assembly guided by a single 2D image [LMS⁺20], or utilizing PQ-NET, a generative seq2seq network that sequentially predicts part poses to form 3D shapes [WZX⁺20].

Although highly successful, these semantic methods rely heavily on prior structural knowledge and fail when applied to real-world objects shattered into meaningless fragments. This necessitated a transition toward **Deep Geometric Fracture Assembly**, which relies purely on the geometric cues of fracture surfaces. A notable method in the space is Neural Shape Mating (NSM) [CLT⁺22], which leverages a Transformer network to predict poses and an adversarial shape prior to evaluate the visual plausibility of the mated geometry (Figure 4.3). However, NSM is strictly specialized for pairwise alignment (assembling exactly two pieces) and struggles to scale to complex multi-piece objects [CLT⁺22].

A major turning point in the literature occurred with the release of the Breaking Bad dataset, a large-scale synthetic dataset that simulates how real-world objects break using physically-based fracture algorithms [S⁺22]. By removing semantic part consistency, this dataset shifted the direction of exploration definitively toward multi-piece fracture assembly. In this context, Jigsaw emerged as a historical milestone. It is widely recognized as the first learning-based framework specifically designed for the assembly of multiple, physically broken 3D pieces. By using a joint-learning pipeline with attention layers to simultaneously optimize point-wise

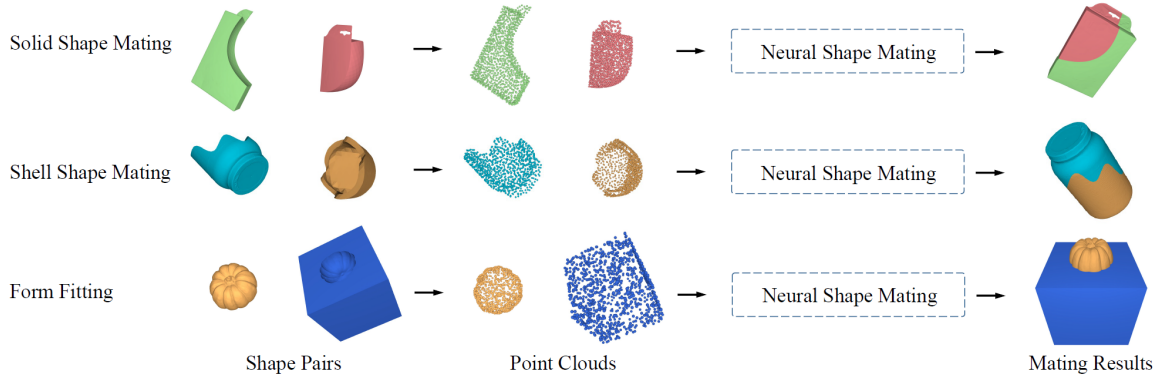


Figure 4.3: The Neural Shape Mating pipeline [CLT⁺22]. A Transformer-based network predicts relative poses for two fragments, using an adversarial shape prior to evaluate the geometric plausibility of the assembled result.

surface segmentation and global multi-part matching, Jigsaw successfully bridged the gap between semantic-free geometric mating and multi-piece global alignment [LSH23].

Consequently, Jigsaw established the standard baseline for subsequent research in this domain. Its pipeline consists of four stages: PointNet++ feature extraction with self-attention and cross-attention, binary fracture surface segmentation, global matching via primal-dual descriptors and Sinkhorn optimal transport, and RANSAC-based pose recovery with Shonan synchronization [LSH23]. The full pipeline is described in detail in Chapter 5. Although Jigsaw established a robust baseline for multi-part assembly, its architecture presented several critical bottlenecks.

4.2 New generation of SOTA

The following section examines the structural bottlenecks in the Jigsaw pipeline and reviews how newer architectures address them. Each subsection concludes by connecting the reviewed ideas to our proposed modifications, establishing the motivation for the extensions presented in Chapter 5.

4.2.1 The Efficiency Bottleneck

First, relying on dense point-wise matching—where every single point on a fragment is compared to every point on another—is computationally expensive, making it difficult to scale to complex objects with many parts.

PHFormer: Multi-Fragment Assembly Using Proxy-Level Hybrid Transformer

To address the computational bottleneck of dense point matching, the PHFormer

[CYD24] model adopts a patch-based representation. Instead of operating on individual points, the input point cloud is partitioned into a set of local regions, referred to as proxies (Figure 4.4). These proxies are constructed by first selecting representative points using Farthest Point Sampling [QYSG17], and then grouping their neighboring points to form local patches. For each proxy, an edge convolution operation is applied to extract a feature embedding that captures its local geometric structure. This transformation reduces the complexity of the matching problem while preserving essential spatial information.

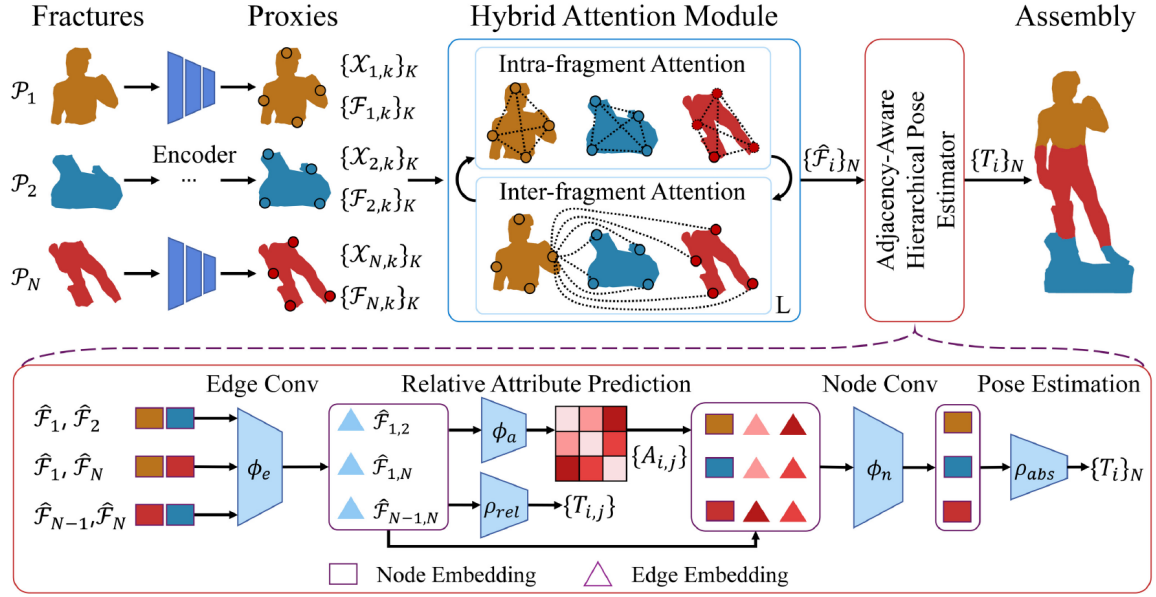


Figure 4.4: The PHFormer architecture [CYD24]. Point clouds are partitioned into proxy-level patches, which are processed through iterative intra- and inter-fragment attention layers before decomposing pose estimation into adjacency prediction and transformation regression.

PHFormer processes these proxy features using a two-level attention mechanism applied iteratively: **intra-fragment attention** builds a coherent representation of each fragment’s internal geometry, while **inter-fragment attention** enables reasoning about spatial relationships across fragments.

Based on these enriched representations, PHFormer decomposes pose estimation into adjacency prediction, relative pose estimation, and absolute pose regression. Fragments form a graph whose edges are weighted by predicted adjacency probabilities, and message passing over this graph produces globally consistent rigid poses.

3D Geometric Shape Assembly via Efficient Point Cloud Matching

A complementary approach to the efficiency problem is the Proxy Match Transformer (PMTR) [LML⁺24], which replaces dense point-to-point comparisons with

a sub-quadratic matching paradigm based on a shared proxy space (Figure 4.5).

The key idea is to introduce a small, learnable **Proxy Tensor** P whose dimensionality is much smaller than either fragment’s feature map. Each fragment independently projects its features onto this shared proxy, and the dot-product of the two projections approximates the full pairwise correlation, but without ever computing the $O(N \times M)$ quadratic matrix. A U-Net-style [RFB15] backbone produces multi-scale feature maps (coarse, medium, fine), and the proxy matching is applied hierarchically: coarse matches identify candidate mating regions, which are refined at finer resolutions. The final correspondences are normalized via Sinkhorn Optimal Transport [Cut13] and used to compute the rigid transformation via Singular Value Decomposition [GVL13].

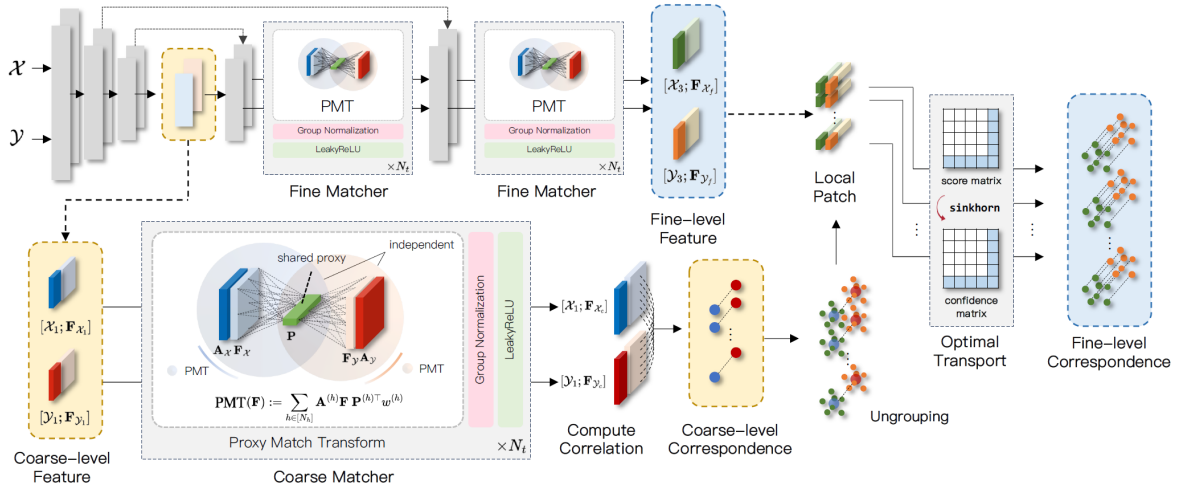


Figure 4.5: The PMTR architecture [LML⁺24]. A coarse-to-fine matching hierarchy uses learned Proxy Tensors to establish sub-quadratic correspondences across multiple spatial resolutions, culminating in SVD-based rigid pose estimation.

While PHFormer and PMTR address the matching cost, a related but distinct efficiency concern lies in the PointNet++ backbone itself: the k -nearest neighbors grouping is topology-unaware and can include spatially close but surface-disconnected points. Inspired by the proxy-level locality of PHFormer, we investigate Gabriel graph filtering [GS69] as a complementary approach that targets the neighborhood construction stage, pruning kNN edges that cross interior space to produce surface-faithful local regions.

4.2.2 The Attention Bottleneck

Jigsaw’s feature extractor applies a single round of self-attention and cross-attention, which limits both *what* information enters the attention computation and *how many* refinement passes the features undergo. On the first aspect, cross-attention relies

purely on learned feature similarity and can produce overconfident correspondence distributions, particularly on out-of-distribution geometry. On the second, a single pass may not suffice for the features to fully capture both intra-fragment structure and inter-fragment relationships. Recent architectures address both aspects.

Geometric Point Attention Transformer for 3D Shape Reassembly

The Geometric Point Attention Transformer (GPAT) [LCML23] addresses the attention bottleneck by explicitly encoding geometric relationships into the attention computation.

The architecture begins with a PointNet backbone that extracts local geometric features (h_i^{local}) from each fragment. These are pooled across all parts to form a global context feature (h^{global}), providing a structural shape prior. The local and global features are concatenated with recycled geometric embeddings (initialized to zero on the first pass) and processed through an MLP to produce node features (h_i). Simultaneously, pairwise cross-part features (z_{ij}) are constructed by concatenating the local features of fragments i and j together with recycled geometric pair features.

To iteratively refine these features, GPAT introduces a specialized attention mechanism composed of three branches:

- **Part Attention** acts as a global context extractor. Standard dot-product attention over the node features (h_i) evaluates high-level interactions between all fragments, forming a fully connected graph where parts attend to one another.
- **Pair Attention** introduces explicit spatial constraints. It transforms the cross-part features (z_{ij}) into edge attention terms, encoding invariant geometric signals: the distances between the centers of mass of each pair of pieces and the triplet-wise dihedral angles between them, mapped into a continuous vector space via Radial Basis Functions (RBFs). This spatial bias is crucial for maintaining geometrically valid relative placements.
- **Point Attention** models 6-DoF pose interactions directly. Each part’s node features are mapped to N virtual 3D keypoints in Euclidean space. The currently estimated rigid poses (T_i, T_j) are applied to these query and key points, and alignment quality is measured via $\|T_i \circ q_i^m - T_j \circ k_j^m\|_2$.

The outputs of the three branches are fused into a single attention score: $a_{ij} = \text{softmax}(n_{ij} + e_{ij} - p_{ij})$. The Point Attention term (p_{ij}) is subtracted because it represents a spatial distance error: larger misalignment penalizes the attention score.

Rather than predicting absolute poses, the network outputs a relative update ($\delta R_i, \delta t_i$) at each iteration. GPAT then employs **Geometric Recycling**: the updated poses are physically applied to the point clouds, which are passed back through the

PointNet backbone to re-extract spatially aware features for the next iteration. This progressive refinement is central to the architecture’s effectiveness: the geometric descriptors become increasingly informative as the fragments approach their correct positions (Figure 4.6).

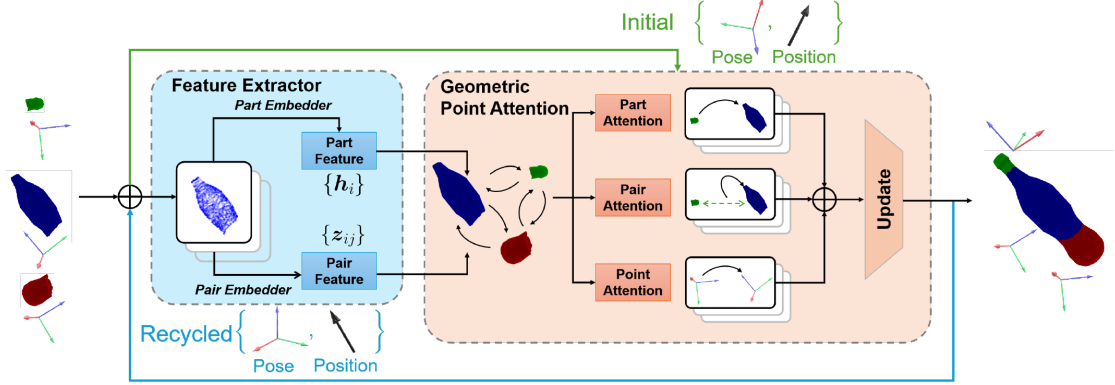


Figure 4.6: Overview of the GPAT architecture [LCML23]. The attention mechanism integrates feature similarity (Part Attention), geometric pair descriptors (Pair Attention), and point-level pose alignment (Point Attention). Geometric Recycling feeds updated poses back through the feature extractor for iterative refinement.

Thus, the pair attention descriptors in GPAT are informative *because* of geometric recycling: the iterative pose updates progressively increase the variance of the centroid distances and triplet angles. In a single-pass pipeline like Jigsaw, where each fragment is independently centered and randomly rotated, these descriptors would have low variance. This observation motivated our investigation of what role the pair attention mechanism takes on when transferred into such a context (Section 5.2.1).

Additionally, both PHFormer and GARF employ an *alternating* strategy of self-attention followed by cross-attention, repeated over multiple rounds (2 in PHFormer, 6 in GARF), allowing each fragment to progressively refine its representation based on global context. This motivated our gated double attention extension, which adds a second round of self- and cross-attention to Jigsaw’s feature extractor (Section 5.2.2).

4.2.3 The Blindness Bottleneck

Classical reassembly methods rely primarily on local geometric cues, such as matching fracture surfaces. While effective in controlled settings, these approaches often fail when fragments are incomplete, eroded, or misaligned, as they lack an explicit representation of the global object structure.

Jigsaw++: Learning Global Shape Priors for Robust Reassembly

Jigsaw++ [LHH24] addresses this limitation by introducing a generative global shape prior (Figure 4.7). The architecture operates in two phases. First, a **Rectified Flow** model is trained on intact objects: their 3D geometry is projected into multi-view images, encoded via LEAP [JJZH24] (a DINOv2-based image-to-3D encoder), and the flow learns to transport Gaussian noise to the latent distribution of complete shapes. Second, the broken object is encoded through the same pipeline, producing an incomplete latent representation. A controlled perturbation via Langevin dynamics pushes this latent into a better starting position, and the Rectified Flow transports it toward the manifold of valid complete shapes.

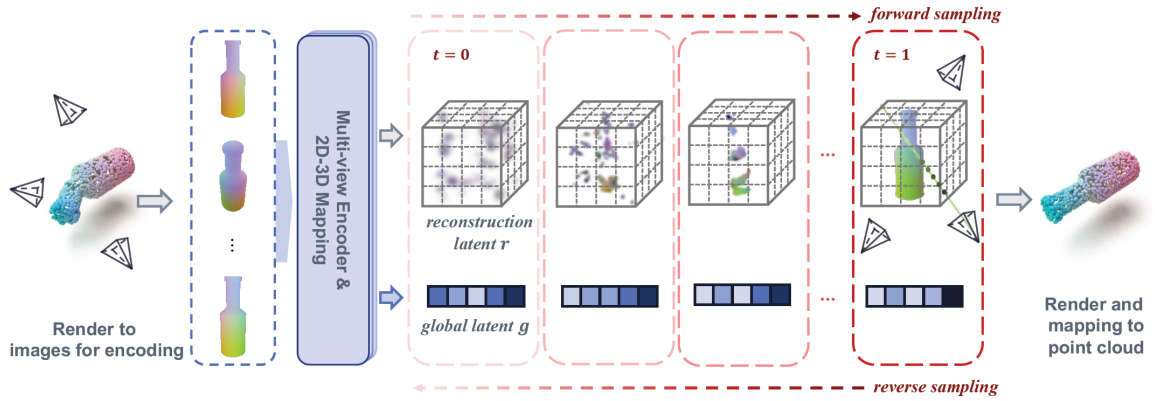


Figure 4.7: Overview of the Jigsaw++ pipeline [LHH24]. Partial assemblies are rendered into multi-view images and encoded into a latent space. Rectified Flow transports the incomplete latent toward the manifold of valid complete shapes, providing a global structural prior for subsequent geometric alignment.

The output is not a final alignment but a coherent global shape reconstruction that provides geometric guidance for subsequent matching, enabling the pipeline to reason about what the assembled object *should* look like even when fragments are incomplete.

4.2.4 The Synthetic-to-Real Gap

The majority of the previous models were trained and tested on the synthetic Breaking Bad dataset. Synthetic fractures lack the complex deterioration, erosion, and extraneous missing parts found in the real world archaeological or medical fractures.

GARF: Flow-Based 3D Reassembly on SE(3)

Unlike prior methods that formulate reassembly as discrete matching or regression, GARF [LJC⁺25] recasts the task as a *continuous dynamical system* on the Lie group

$SE(3)$ (Figure 4.8). Instead of predicting final poses directly, the model learns a velocity field that moves fragments from a random initial configuration toward the ground-truth assembly. Each fragment pose $T^i = (r^i, a^i) \in SE(3)$ evolves according to:

$$\frac{d}{dt}\psi_t(T) = v_t(\psi_t(T)), \quad v_t \in \mathfrak{se}(3), \quad (4.1)$$

where $\mathfrak{se}(3)$ is the tangent space encoding instantaneous rigid motion. The rotation and translation components are decoupled and updated independently via the exponential map:

$$r_{t+\Delta t}^i = \exp(\Delta t \cdot \hat{\omega}_t^i) \cdot r_t^i, \quad (4.2)$$

$$a_{t+\Delta t}^i = a_t^i + \Delta t \cdot \hat{v}_t^i, \quad (4.3)$$

where $\exp(\cdot)$ maps from $\mathfrak{so}(3)$ to $SO(3)$. The network is trained via a flow-matching objective that aligns predicted velocities with ground-truth geodesics on $SE(3)$.

Feature extraction uses a Point Transformer V3 (PTv3) backbone with Poisson disk resampling to prevent density memorization. The features are processed through a deep Transformer that alternates **intra-fragment self-attention** (implemented via FlashAttention ??) with **global cross-attention** over 6 rounds, progressively refining each fragment’s representation. To resolve global ambiguity, a random subset of fragments is frozen as *anchors* at each training iteration, forcing the network to learn relative rather than absolute positions. At inference, a one-step coarse alignment collapses the search space, followed by multi-step ODE integration for refinement.

To bridge the synthetic-to-real gap, GARF uses few-shot domain adaptation via LoRA [?]: trainable low-rank adapters are injected only into the attention layers, allowing the model to adapt with as few as 5 to 10 real-world objects. The authors introduce the FRACTURA dataset, containing real-world fractures from ceramics, bones, eggshells, and lithics, as the first systematic benchmark for real-world fracture assembly. GARF achieves state-of-the-art performance on both Breaking Bad and FRACTURA.

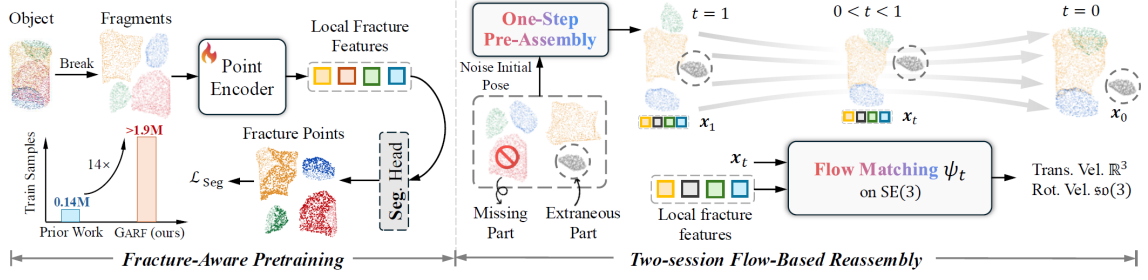


Figure 4.8: Overview of the GARF pipeline [LJC⁺25]. Fragment poses evolve along learned velocity fields on $SE(3)$, from an initial noisy configuration (left) to a coherent assembly (right) through a two-stage coarse-to-fine integration scheme.

4.2.5 Summary and Research Gap

Each of the reviewed architectures introduces ideas that address specific bottlenecks in the Jigsaw pipeline. Rather than proposing an entirely new architecture, this thesis asks which of these ideas can be extracted, adapted, and integrated into Jigsaw as modular extensions. The specific connections are:

- **Pair attention as learned regularization** (inspired by GPAT). We studied the pair attention mechanism from GPAT and recognized that, without geometric recycling, the spatial descriptors would have low variance in Jigsaw’s preprocessing context. We hypothesized that the mechanism would act as a learned attention regularizer rather than as a discriminative spatial bias, and designed the experiment to test this.
- **Gated double attention** (inspired by PHFormer and GARF). Both PHFormer and GARF employ alternating rounds of self-attention and cross-attention, progressively refining fragment representations. We adopt this principle by adding a second gated attention pass to Jigsaw’s feature extractor.
- **Gabriel graph filtering** (inspired by PHFormer and PMTR). While PHFormer and PMTR address the matching cost through proxy-level representations, a related bottleneck lies in PointNet++’s topology-unaware kNN grouping. We apply Gabriel graph filtering [GS69] to prune edges that cross interior space, producing surface-faithful neighborhoods.
- **ICP refinement**. Jigsaw recovers pairwise poses from sparse RANSAC correspondences, while GARF bypasses discrete matching entirely through continuous pose estimation. We take a simpler, non-learned approach: dense point-

to-point ICP refinement of the RANSAC estimates before global synchronization via Shonan averaging.

In the next chapter, we present the extended methodology and analyze how these modifications affect the overall reconstruction, with a focus on challenging scenarios such as incomplete or ambiguous fragment configurations.

Chapter 5

Methodology: Proposed Extensions to Jigsaw

This chapter presents the methodological foundation of this thesis: four targeted extensions to the Jigsaw framework for 3D fractured object reassembly. The original Jigsaw pipeline is first summarized to establish the baseline architecture and identify the insertion points of the proposed modifications. The chapter then describes the four contributions investigated in this work: a pair geometric attention bias repurposed as a learned soft regularizer for cross-attention, a gated double attention module for controlled feature refinement, a Gabriel graph neighborhood filter for surface-faithful feature extraction, and an ICP refinement step for improving pair-wise pose estimates before global synchronization.

5.1 Baseline Pipeline and Modification Points

As introduced in Chapter 4, Jigsaw is a hybrid learning-based framework for multi-piece 3D fracture assembly that combines deep learning with classical geometric algorithms. Its end-to-end pipeline (Figure 5.1) consists of four modules: a front-end feature extractor (PointNet++ backbone with self-attention and cross-attention layers), a surface segmentation module that classifies points as fracture or intact, a multi-part matching module that establishes global point-to-point correspondences via primal-dual descriptors and Sinkhorn optimal transport, and a pose recovery module that estimates pairwise transformations via RANSAC and synchronizes them globally via Shonan averaging [DRW⁺20].

The proposed extensions target three specific points in this pipeline, summarized in Table 5.1. The remaining sections describe the baseline components in enough detail to explain the integration of each modification, followed by the modifications themselves.

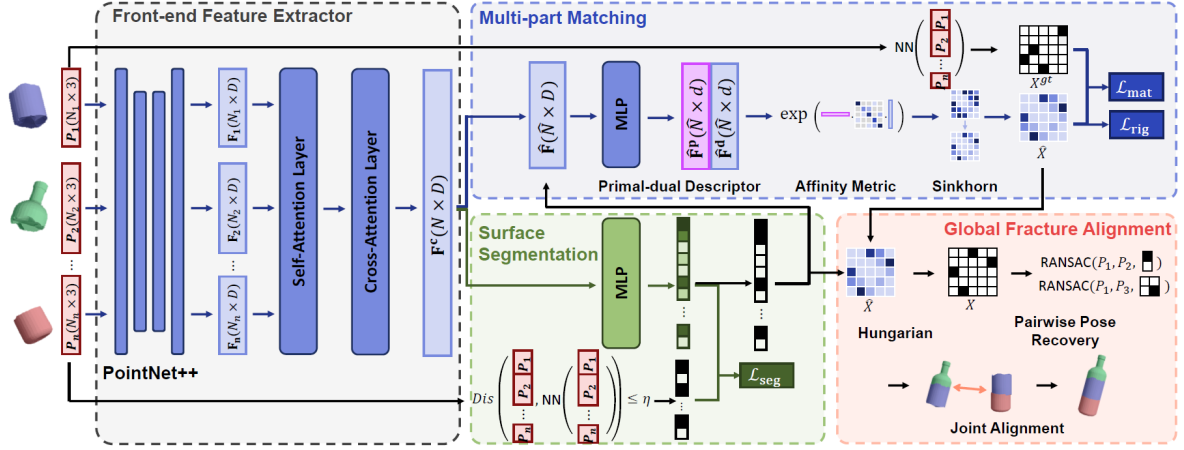


Figure 5.1: End-to-end pipeline of the Jigsaw framework [LSH23]. Point clouds are processed through a PointNet++ backbone with attention layers (feature extraction), classified into fracture and intact surfaces (segmentation), globally matched using primal-dual descriptors with Sinkhorn optimal transport (matching), and aligned via RANSAC and Shonan averaging (pose recovery).

Table 5.1: Overview of the proposed modifications and their integration points in the Jigsaw pipeline.

Pipeline Stage	Original Component	Proposed Modification
Feature extraction	Cross-attention	Pair attention bias (soft regularization)
Feature refinement	Single attention pass	Gated double attention pass
Neighborhood construction	kNN grouping	Gabriel graph filtering
Pose recovery	RANSAC pairwise estimation	ICP refinement before synchronization

5.2 Front-End Feature Extractor

Baseline component. Jigsaw uses a PointNet++ backbone (see Section 3.2) to extract per-point geometric features at multiple scales, followed by attention layers that model relationships both within and across fragments. PointNet++ is chosen over PointNet because its hierarchical architecture (Farthest Point Sampling, local neighborhood grouping, and recursive PointNet applications) preserves the fine-grained local geometry of fracture surfaces, which is critical for matching complementary fragments.

While PointNet++ provides strong static embeddings for local geometric regions, fractured object reassembly also requires modeling how these features relate to one another, both within the same fragment and across different fragments. This relational context is captured through two attention layers.

First, a Point Transformer self-attention layer models the internal structure of

each fragment:

$$f_p^s = \sum_{f_q \in N(p)} \text{softmax}(\text{MLP}_s((W_Q^s f_p - W_K^s f_q + p^{enc})) \odot (W_V^s f_q + p^{enc})) \quad (5.1)$$

The attention is restricted to the k -nearest neighbors for efficiency. Unlike standard scalar attention, MLP_s produces vector-valued weights enabling per-channel modulation ($\odot$), and relative positional encodings p^{enc} make the representation sensitive to physical displacement between points.

After encoding each fragment’s internal structure, a **cross-attention** layer models inter-fragment relationships. In this layer, queries are obtained from one fragment while keys and values come from other fragments, allowing local features on one fracture surface to interact with features from other fragments. This supports the discovery of point-to-point correspondences required for reassembly. A standard multi-head attention block is applied:

$$F^c = \text{FFN}(W_O^m(\text{cat}(\text{head}_1, \dots, \text{head}_n))) \quad (5.2)$$

where

$$\text{head}_i = \text{Attention}(W_{Q_i}^m F^s, W_{K_i}^m F^s, W_{V_i}^m F^s) \quad (5.3)$$

and

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (5.4)$$

Here, $\text{FFN}(\cdot)$ consists of two linear layers with ReLU and LayerNorm. This cross-attention layer is the insertion point for the pair geometric attention bias (Section 5.2.1), and the entire self-attention + cross-attention block is duplicated by the gated double attention extension.

5.2.1 Proposed Modification: Pair Geometric Attention as Soft Regularization

Motivation. The GPAT architecture [LCML23] introduces a **Pair Attention** mechanism that adds a learned geometric bias b_{ij} to cross-attention logits for each fragment pair (i, j) . The bias is computed from spatial descriptors (centroid distances and triplet angles) encoded via Gaussian Radial Basis Functions (RBFs) and projected through a small MLP:

$$\alpha_{pq} = \frac{\mathbf{q}_p^T \mathbf{k}_q}{\sqrt{d_k}} + b_{ij}, \quad b_{ij} = \text{MLP}([\text{RBF}(d_{ij}); \sum_k \text{RBF}(\cos \angle_{ijk})]). \quad (5.5)$$

In GPAT, these descriptors carry rich geometric information because the architecture maintains a globally consistent coordinate frame and iteratively refines fragment poses via geometric recycling, so that d_{ij} and $\angle_{ijk}$ become progressively more informative with each iteration.

When studying this mechanism, we recognized that Jigsaw’s preprocessing would fundamentally alter its behavior. In Jigsaw, each fragment is independently centered at the origin and randomly rotated, which reduces the variance of the geometric descriptors: pairwise centroid distances are close to zero and triplet angles follow a near-uniform distribution. We hypothesized that rather than providing discriminative spatial guidance, the pair encoder would learn an approximately *uniform additive bias* across all fragment pairs, equivalent to a learned increase of the softmax temperature. Unlike a fixed temperature hyperparameter, however, this bias would be *data-dependent*: the MLP would adjust its magnitude based on the training signal, acting as a **soft attention regularizer** that prevents overconfident correspondences, particularly on out-of-distribution geometry where the base attention is poorly calibrated.

Architecture. The pair encoder has two stages. First, the pairwise centroid distances and triplet angles are mapped to continuous feature vectors through Gaussian RBF layers whose centers are fixed and do not change during training. Second, a two-layer MLP (`bias_proj`) projects the concatenated RBF features to the scalar bias b_{ij} that is added to the cross-attention logits (Equation 5.5). Only the MLP parameters are learned; the RBF encoding remains identical to its pretrained GPAT initialization. The encoder is connected to the cross-attention layer through the same zero-initialized gating strategy as the double attention module (Section 5.2.2), so the bias starts at zero and grows only if the training objective benefits from it.

Expected Benefits. The smoothing is expected to primarily benefit out-of-distribution generalization, where attention distributions are less reliable and prone to overconfident mismatches. On in-distribution data, the base cross-attention is already well-calibrated, so the additional smoothing may provide little benefit.

Limitations. In Jigsaw’s preprocessing context, the pair encoder operates as a regularizer rather than as the discriminative spatial module it was designed to be in GPAT. In multi-piece settings, the uniform bias is spread across a larger attention matrix and its regularization effect may be diluted.

5.2.2 Proposed Improvement: Gated Double Attention

The original Jigsaw architecture applies a single self-attention layer followed by a cross-attention layer to extract and propagate geometric features. While this design enables both intra-fragment and inter-fragment reasoning, it limits the model to a single round of feature refinement before downstream tasks such as segmentation and matching.

Recent work suggests that relational reasoning benefits from multiple iterative passes. For example, PHFormer repeatedly applies intra-fragment and inter-fragment attention to progressively refine proxy-level features [CYD24], while PMTR stacks multiple matching layers that first establish coarse correspondences and then refine them into precise alignments [LML⁺24]. These observations motivate extending Jigsaw with an additional attention pass, allowing the model to revisit and refine its feature representations.

However, unlike these approaches, which are trained from scratch, our setting starts from a pretrained Jigsaw model. Directly adding a second attention block with randomly initialized weights may disrupt the learned feature distribution during finetuning. To address this, we introduce the second attention pass using **zero-initialized gated residual connections**, inspired by the ReZero mechanism [BMM⁺] (Figure 5.2). This design allows the model to initially behave identically to the pre-trained network, while gradually incorporating additional capacity during training.

Architecture. Let $\mathbf{f}^{(1)}$ denote the feature representation produced by the original attention block. We introduce a second self-attention layer followed by a second cross-attention layer, whose contributions are modulated by learnable scalar gates:

$$\mathbf{f}_{\text{self}}^{(2)} = \mathbf{f}^{(1)} + g_s \cdot (\text{SelfAttn}_2(\mathbf{f}^{(1)}) - \mathbf{f}^{(1)}), \quad (5.6)$$

$$\mathbf{f}_{\text{cross}}^{(2)} = \mathbf{f}_{\text{self}}^{(2)} + g_c \cdot (\text{CrossAttn}_2(\mathbf{f}_{\text{self}}^{(2)}) - \mathbf{f}_{\text{self}}^{(2)}). \quad (5.7)$$

Here, g_s and g_c are learnable scalar parameters initialized to zero. Both SelfAttn_2 and CrossAttn_2 share the same architecture as their first-pass counterparts, but use independent parameters. At initialization, $g_s = g_c = 0$ and the second pass is inactive ($\mathbf{f}_{\text{cross}}^{(2)} = \mathbf{f}^{(1)}$), so the model starts from the pretrained solution. During training, the gates are free to grow, incorporating the second pass only if it improves the objective.

Expected Benefits. The additional self-attention layer enables further refinement of local geometric features within each fragment, while the second cross-attention layer provides an additional opportunity for information exchange between frag-

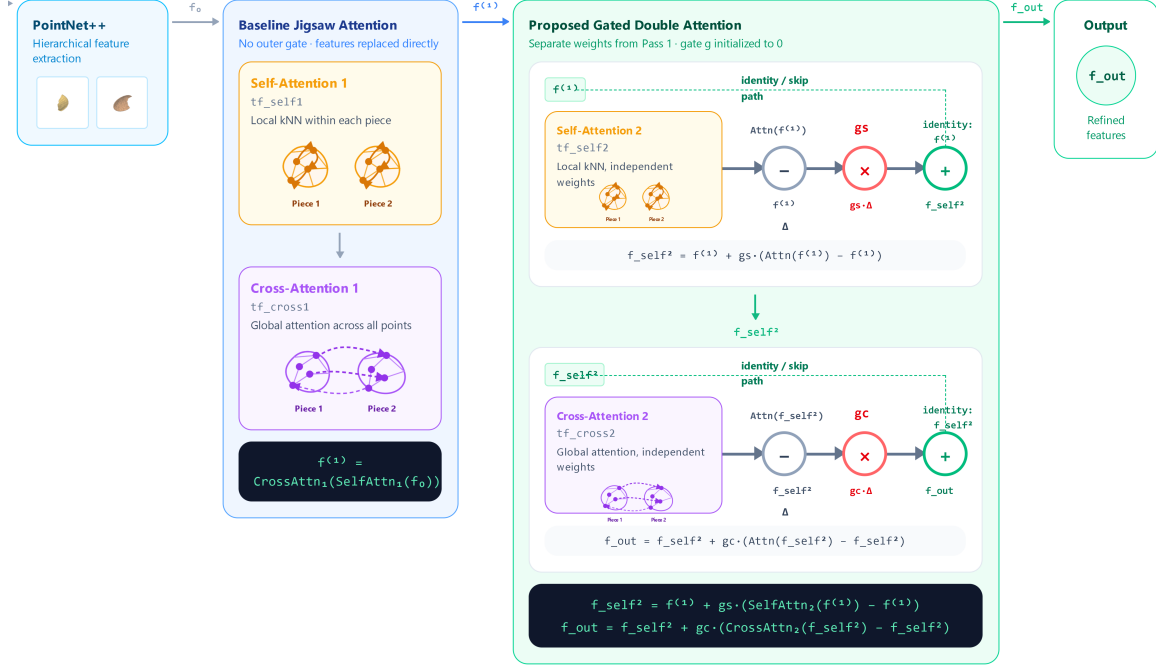


Figure 5.2: Gated double attention architecture. The baseline Jigsaw attention (left) applies a single pass of self-attention and cross-attention. The proposed extension (right) adds a second pass with independently initialized weights, connected via zero-initialized ReZero gates (g_s, g_c) that allow the network to gradually incorporate the second pass during training.

ments. Together, they act as a refinement stage that can improve both intra-fragment consistency and inter-fragment relationships.

Limitations. The proposed modification increases computational cost and model complexity. In addition, since the contribution of the second attention pass is controlled by learned gates, it may have little effect if the optimization does not find it beneficial.

5.2.3 Proposed Extension: Gabriel Graph Neighborhood Filtering

Motivation. The PointNet++ backbone constructs local neighborhoods using k -nearest neighbors (kNN), which groups the K spatially closest points around each centroid regardless of surface topology. On dense point clouds, kNN can include spatially close points that are not surface-connected to the centroid, for example points on a nearby but distinct surface region, or points reachable only by passing through the object’s interior. These topologically distant neighbors introduce geometrically misleading information into the shared MLP and max-pooling aggregation, potentially degrading the quality of the learned local features.

Gabriel Graph Criterion. The Gabriel graph [GS69], originally introduced for geographic contiguity analysis, provides a parameter-free geometric test for surface-faithful connectivity. Two points A and B are connected in the Gabriel graph if and only if no other point C lies inside the open ball whose diameter is the segment $\overline{AB}$:

$$(A, B) \in \mathcal{G} \iff \forall C \neq A, B : \|C - \frac{A+B}{2}\|^2 > \frac{\|A-B\|^2}{4} \quad (5.8)$$

Intuitively, if a third point falls inside this ball, it lies “between” A and B , indicating that the edge $\overline{AB}$ likely crosses interior space rather than following the surface. By filtering kNN neighborhoods through this criterion, we retain only edges that respect the local surface geometry (Figure 5.3). As a side effect, reducing the number of neighbors that participate in feature aggregation lowers the computational cost of the shared MLP and max-pooling operations at each Set Abstraction level.

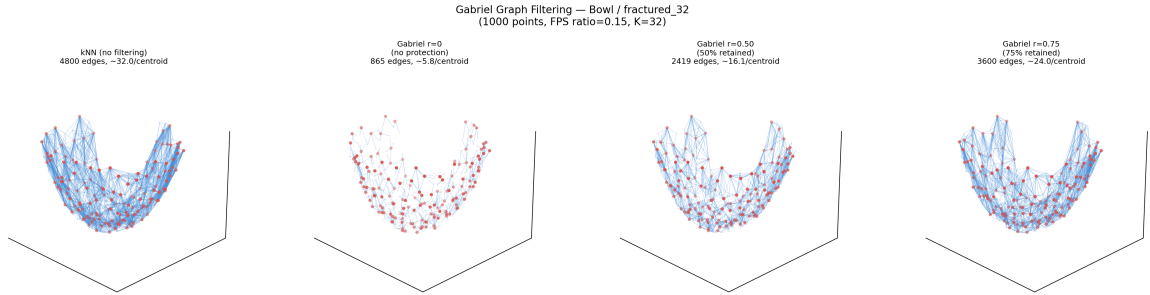


Figure 5.3: Gabriel graph filtering applied to a fractured bowl. From left to right: standard kNN (4800 edges, ~ 32 per centroid), Gabriel $r=0$ with no protection (865 edges, ~ 5.8 per centroid), Gabriel $r=0.50$ (2419 edges, ~ 16.1 per centroid), and Gabriel $r=0.75$ (3600 edges, ~ 24.0 per centroid). The filter progressively removes edges that cross interior space while preserving surface-faithful connectivity.

Integration into PointNet++. The Gabriel filter is applied after kNN grouping and before the shared MLP at each Set Abstraction level. For each centroid, every neighbor is tested against the Gabriel criterion using the remaining neighbors as potential witnesses. Neighbors that fail the test are excluded from the feature aggregation.

Preliminary experiments revealed that the Gabriel criterion is highly aggressive on dense point clouds: with $K=32$, naive filtering retained only ~ 6 neighbors per centroid (19% retention). To ensure sufficient diversity for max-pooling, we protect the closest $\lfloor K \cdot r \rfloor$ neighbors unconditionally and apply Gabriel filtering only to the remaining outer portion:

$$\text{filtered}(j) = \begin{cases} \text{keep} & \text{if } j \leq \lfloor K \cdot r \rfloor \\ \text{Gabriel test} & \text{otherwise} \end{cases} \quad (5.9)$$

where $r \in [0, 1]$ is the minimum keep ratio. The filter introduces no learnable parameters; it is a purely geometric operation that does not change the model’s parameter count.

Expected Benefits. By removing topologically distant neighbors, the local features computed via max-pooling should more faithfully represent the surface geometry around each centroid, potentially improving the quality of downstream point correspondences and pose estimation.

Limitations. The choice of r controls the trade-off between surface fidelity and neighborhood diversity: too aggressive filtering leaves max-pooling with insufficient diversity, while too conservative filtering has negligible effect. Additionally, the filter’s aggressiveness depends on local point density, which varies across objects and may introduce sensitivity to the data distribution.

5.3 Fracture Surface Segmentation

Unmodified baseline component. This module is inherited from Jigsaw without modification, but is described here because its output defines the input to both the matching module and the ICP refinement step.

Following feature extraction, the network classifies each point as fracture or intact surface. An MLP maps the extracted feature vector f_p to a fracture confidence score $\tilde{c}_p \in [0, 1]$ via Sigmoid activation, trained with binary cross-entropy:

$$L_{seg} = -\frac{1}{N} \sum_{p \in O} [c_p \log \tilde{c}_p + (1 - c_p) \log(1 - \tilde{c}_p)] \quad (5.10)$$

Ground truth labels are generated automatically: a point is labeled as fracture ($c_p = 1$) if its distance to the nearest point on any other fragment is below $\eta = 0.025$:

$$c_p = \begin{cases} 1 & \text{if } \text{Dis}(p, \text{NN}(p, P \setminus P_i)) \leq \eta \\ 0 & \text{otherwise} \end{cases} \quad (5.11)$$

By isolating fracture points, this module reduces the search space for the subsequent matching stage to the actual broken boundaries.

5.4 Multi-Part Point Matching

Unmodified baseline component. This module is inherited from Jigsaw without modification. It is summarized here because its output (point-to-point correspondences) feeds directly into the pose recovery stage, where the proposed ICP refinement operates.

Unlike sequential pairwise methods that accumulate drift, Jigsaw evaluates all fracture points from all fragments simultaneously, using optimal transport and assignment to predict globally consistent correspondences.

Primal-Dual Descriptors. When a solid object breaks, the resulting pieces exhibit complementary geometry: a convex protrusion on one fragment corresponds to a concave indentation on another. Standard feature extractors are designed to find identical surfaces, which fails in this context. Jigsaw addresses this by projecting the fracture surface embeddings $\hat{f}_i^c$ through an MLP (1×1 convolution with ReLU and normalization) into two complementary spaces (Figure 5.4): a **Primal Descriptor** $\hat{f}_i^p$ encoding the local convex shape, and a **Dual Descriptor** $\hat{f}_i^d$ encoding the corresponding concavity.

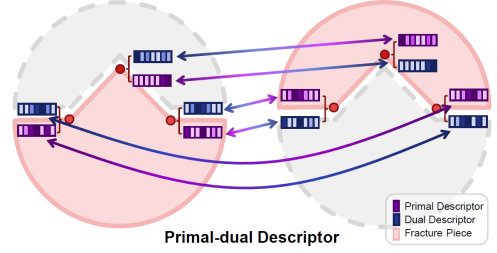


Figure 5.4: The Primal-Dual Descriptor mechanism. Fracture features are projected into complementary primal (convex) and dual (concave) spaces.

into two complementary spaces (Figure 5.4): a **Primal Descriptor** $\hat{f}_i^p$ encoding the local convex shape, and a **Dual Descriptor** $\hat{f}_i^d$ encoding the corresponding concavity.

Global Affinity and Sinkhorn Optimal Transport. A global affinity matrix M is computed via the scaled dot-product between primal and dual features:

$$M = \exp\left(\frac{\hat{F}_p^T A \hat{F}_d}{\tau}\right) \quad (5.12)$$

where A represents learnable affinity weights and τ is a temperature parameter. A **Sinkhorn layer** [Cut13] converts these scores into a doubly stochastic matrix $\tilde{X}$ by iteratively normalizing rows and columns, enforcing a soft one-to-one matching constraint that approximates a permutation matrix.

Training Losses. The matching loss is a cross-entropy between predicted soft matches and a ground-truth correspondence matrix X_{gt} :

$$\mathcal{L}_{mat} = -\frac{1}{N} \sum_{1 \leq i, j \leq \tilde{N}} [x_{ij}^{gt} \log \tilde{x}_{ij} + (1 - x_{ij}^{gt}) \log(1 - \tilde{x}_{ij})] \quad (5.13)$$

The ground truth is constructed automatically: $x_{ij}^{gt} = 1$ if point j is the nearest neighbor of point i on a different fragment.

Even if the network correctly predicts which points correspond to one another, those matches might not translate into a physically valid rigid transformation (rotation and translation without warping). To enforce this, Jigsaw applies a **Rigidity Loss**. For each fracture point p on fragment i , the matching matrix $\tilde{X}_{ij}$ predicts a target location p' on fragment j by computing a probability-weighted average of all matchable points:

$$p' = \frac{\tilde{X}_{ij}(p)P_j}{\|\tilde{X}_{ij}(p)\|_1} \quad (5.14)$$

The optimal rigid transformation is obtained by solving the orthogonal Procrustes problem differentially via SVD:

$$R_{ij}, t_{ij} = \underset{R \in SO(3), t \in \mathbb{R}^3}{\operatorname{argmin}} \sum_{p \in P_i} \|\tilde{X}_{ij}(p)\|_1 \|Rp + t - p'\|_2^2 \quad (5.15)$$

The Rigidity Loss $\mathcal{L}_{rig}$ penalizes this residual alignment error, ensuring the predicted point matches are consistent with a valid physical transformation. The total training objective combines all three losses:

$$L = \alpha L_{seg} + \beta L_{mat} + \gamma L_{rig} \quad (5.16)$$

Hungarian Assignment. At inference time, the continuous Sinkhorn output $\tilde{X}$ is discretized by the **Hungarian algorithm**, which solves the linear assignment problem in polynomial time to produce globally optimal 1-to-1 correspondences.

5.5 Pose Recovery and Alignment

Baseline component with proposed refinement. The pose recovery pipeline is inherited from Jigsaw, with one proposed modification: an ICP refinement step inserted between RANSAC and Shonan averaging.

Pairwise Pose Recovery via RANSAC. Once the Hungarian algorithm outputs a binary correspondence matrix, the next objective is to determine the rigid transformation (rotation and translation) required to align each pair of fragments. Because the network inevitably predicts some incorrect matches, computing a transformation from all points simultaneously would yield an inaccurate alignment. RANSAC addresses this by repeatedly sampling 3 corresponding point pairs, computing a candidate transformation via SVD-based Procrustes, and evaluating how many remaining pairs align within a predefined error tolerance. The transformation supported by the largest consensus set of inliers is selected, effectively discarding outlier correspondences.

Global Alignment via Transformation Synchronization. While RANSAC suc-

cessfully aligns fragments in isolated pairs, assembling a multi-part object from pairwise transformations alone would lead to cumulative drift. To achieve a globally consistent assembly, the framework represents the object as a factor graph where nodes are the absolute global poses of each fragment and edges are the relative pairwise transformations from RANSAC. Because a single piece may connect to multiple fragments, chaining these relative transformations often results in slight mathematical contradictions (cycle inconsistencies).

To resolve these conflicting constraints, the framework employs Shonan averaging [DRW⁺20] (Figure 5.5), a transformation synchronization algorithm. Rather than aligning pieces one by one, Shonan averaging jointly optimizes the global poses of all nodes simultaneously. It distributes and minimizes the accumulated alignment errors across all edges, finding the solution that best satisfies all local pairwise constraints at once. This results in a cohesive, drift-free final 3D reassembly.

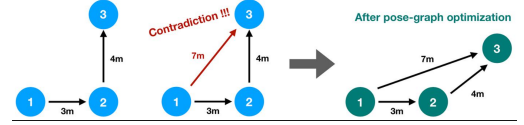


Figure 5.5: Transformation synchronization via Shonan averaging. Pairwise transformations (edges) are jointly optimized to recover globally consistent poses (nodes).

5.5.1 Proposed Improvement: ICP Refinement for Pairwise Poses

The pairwise transformations estimated via RANSAC serve as the input constraints for the global alignment stage. While these transformations are robust to outliers, they are computed from sparse correspondences and therefore only approximate the true geometric alignment between fragments. To further refine these pairwise estimates before global synchronization, we introduce an Iterative Closest Point (ICP) step between RANSAC and Shonan Averaging.

Pipeline Integration. The refinement is applied to every fragment pair for which RANSAC successfully estimates a transformation (i.e., pairs with at least 3 inlier correspondences). Given such a pair, RANSAC produces an initial rigid transformation T_{ransac} , which is then used to initialize ICP:

$$T_{\text{refined}} = \text{ICP}(P_{\text{source}}, P_{\text{target}}, T_{\text{ransac}}). \quad (5.17)$$

The resulting refined transformations replace the RANSAC estimates as input edges in the factor graph optimized by Shonan Averaging.

Motivation. RANSAC and ICP address complementary aspects of the alignment problem (Figure 5.6). RANSAC handles outlier correspondences but relies on small random subsets (3 points per hypothesis) without optimizing over the full surface

geometry. ICP performs local refinement by iteratively minimizing distances between nearby points, but requires a sufficiently accurate initialization. By initializing ICP with the RANSAC estimate, the method combines robustness to outliers with dense geometric refinement.

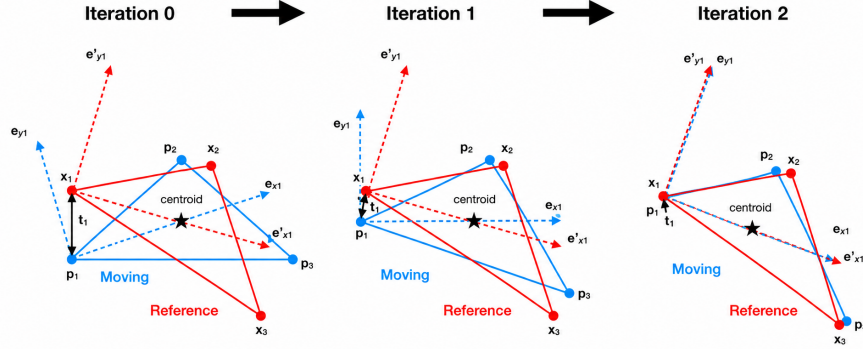


Figure 5.6: Iterative Closest Point (ICP) refinement. At each iteration, points on the moving fragment (blue) are matched to their nearest neighbors on the reference fragment (red), and the rigid transformation that minimizes the sum of squared distances is applied. The process converges as the two point sets progressively align.

Implementation Details. We use the `registration_icp` implementation from Open3D [ZPK18]. The specific configuration is as follows:

- **Variant:** Point-to-point ICP (`TransformationEstimationPointToPoint`), which minimizes the sum of squared distances between closest-point pairs. We do not use point-to-plane ICP, as reliable surface normals are not available from the sparse predicted fracture points.
- **Input points:** Only the predicted fracture surface points (critical points) are used, not the full surface point clouds. This focuses the alignment on the geometrically informative fracture boundaries, where the complementary geometry between fragments provides the strongest alignment signal.
- **Maximum correspondence distance:** 0.05 (in the unit-normalized coordinate system). Point pairs farther apart than this threshold are excluded from the optimization, preventing distant outlier points from distorting the alignment.
- **Scaling:** Disabled (`with_scaling=False`). Since all meshes are pre-normalized to a unit bounding box, the transformation is restricted to pure rotation and translation.
- **Convergence:** Open3D's default criteria are used: the algorithm terminates when the relative change in fitness falls below 10^{-6} , or after 30 iterations, whichever comes first.

Expected Benefits. Since ICP introduces no learnable parameters, it is orthogonal to the attention-based modifications and carries no risk of overfitting. The refinement is expected to consistently tighten pairwise alignments, improving the quality of the constraints fed to global synchronization.

Limitations. The additional ICP step increases computational cost and remains dependent on the quality of the initial transformation. Since ICP operates only on the predicted fracture points rather than the full surface, its effectiveness depends on the quality of the upstream segmentation. Furthermore, point-to-point ICP converges more slowly than point-to-plane ICP and may settle in local minima if the initial alignment is poor. In practice, however, the RANSAC initialization is sufficiently accurate for the refinement to consistently improve alignment quality (see Section 6.4).

Chapter 6

Experiments and Results

This chapter presents the experimental evaluation of the proposed modifications to the Jigsaw framework. We first evaluate all modifications on the full Breaking Bad dataset, then conduct a data scaling analysis by training on a reduced subset ($\sim 63\%$ of the everyday training patterns) to study how the proposed extensions behave under data scarcity. The proposed modifications, particularly gated double attention, provide consistent improvements when data is limited, acting as implicit regularizers that extract geometric patterns a single-pass model cannot recover from sparse examples. The data scaling analysis additionally reveals that **data quantity has a large compounding effect**, with the full dataset yielding over 10 percentage points more than the reduced subset. A further finding is that loss scheduling during fine-tuning has a surprisingly large effect on the gated double attention layers.

6.1 Experimental Setup

6.1.1 The Breaking Bad Dataset

All experiments are conducted on the Breaking Bad dataset [S⁺22], a large-scale benchmark for 3D fracture assembly containing over one million fractured objects simulated from approximately 10,000 base models. Unlike earlier shape assembly datasets such as PartNet [MZC⁺19] or AutoMate [JHC⁺21], which decompose objects along semantically meaningful boundaries (effectively modeling *construction*), Breaking Bad models the *destruction* process: how geometric objects naturally break into fragments under physical impact.

Dataset construction. The base shapes are collected from PartNet (20 daily object categories) and Thingi10K [ZJ16] (sculptures and archaeological-style artifacts). Each mesh is rescaled to a unit bounding box and fractured using the physically-based simulation of Sellán et al. [S⁺22], producing 100 unique fracture patterns per

base model (20 fracture modes + 80 random impacts). Patterns with fewer than 2 or more than 100 pieces are discarded.

Dataset subsets. The dataset is organized into three subsets:

- **Everyday:** 20 categories of daily objects (BeerBottle, Bottle, Bowl, Cookie, Cup, DrinkBottle, DrinkingUtensil, Mirror, Mug, PillBottle, Plate, Ring, Spoon, Statue, Teacup, Teapot, ToyFigure, Vase, WineBottle, WineGlass), sourced from PartNet. Contains 34,075 training and 7,679 validation fracture patterns from 407 and 91 objects respectively.
- **Artifact:** Sculptures and archaeological-style objects from Thingi10K. Contains 3,651 fracture patterns from 40 objects. Serves as a cross-domain generalization benchmark: models are trained exclusively on everyday objects and evaluated on artifacts without any finetuning.
- **Other:** Remaining Thingi10K meshes (video game assets, fabrication models, etc.), which we do not use in our experiments.

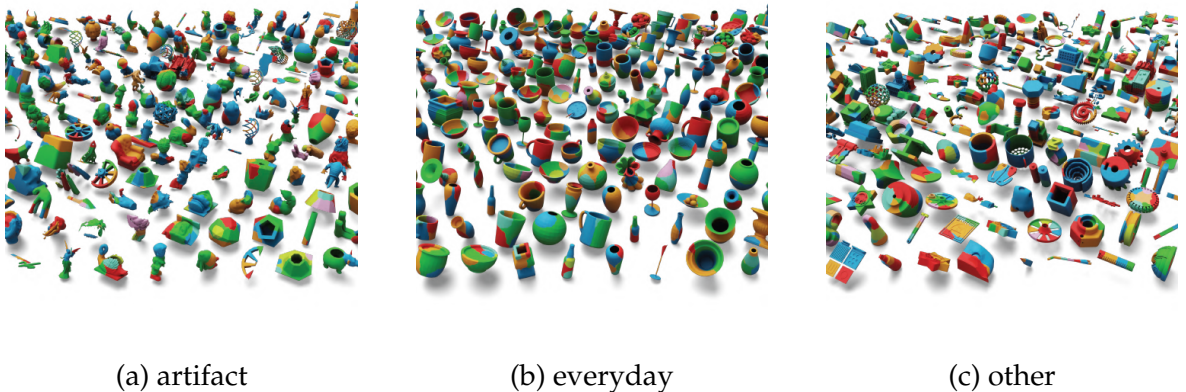


Figure 6.1: Examples of fractured objects from each subset of the Breaking Bad dataset. **(a)** Artifact objects (archaeological sculptures). **(b)** Everyday objects (bottles, bowls, mugs, etc.). **(c)** Other objects (video game assets, fabrication models).

Figure 6.1 shows representative examples from each subset. Following the evaluation protocol of Jigsaw [LSH23], we train on the training split and report all results on the validation split.

6.1.2 Evaluation Metrics

We adopt the standard evaluation metrics used across the 3D assembly literature [LSH23, S⁺22]:

- **Chamfer Distance (CD ↓):** The average bidirectional distance between the predicted and ground-truth point clouds ($\times 10^{-3}$). For each fragment, CD measures how closely the predicted pose aligns the piece to its true position; lower values indicate tighter geometric alignment.
- **Part Accuracy (PA ↑):** The fraction of fragments whose Chamfer distance falls below a threshold $\tau = 0.01$. A fragment is considered correctly assembled if and only if its CD is below this threshold. PA is therefore a *derived* metric that discretizes the continuous CD into a binary correct/incorrect decision per fragment, providing an intuitive overall success rate. This is the primary metric for assembly quality.
- **Rotation MAE (↓):** Mean absolute error of the predicted rotation matrices, measured in degrees.
- **Translation MAE (↓):** Mean absolute error of the predicted translation vectors.

All results are reported as the mean $\pm$ standard deviation over three independent evaluation runs with different random seeds to ensure statistical reliability.

6.1.3 Implementation Details

All models are fine-tuned from the official pretrained Jigsaw checkpoint, which was originally trained for 250 epochs on the full everyday subset (2 to 20 pieces) using Adam with a learning rate of 10^{-3} and cosine annealing. Table 6.1 summarizes the shared hyperparameters used across all finetuning experiments. The only differences between configurations are the architectural modifications (pair attention, double attention) and the loss scheduling, which are noted explicitly in each experiment.

Finetuning strategy. We employ full finetuning with gated residual integration of new modules. All pretrained parameters remain trainable but receive a $10\times$ lower learning rate (10^{-4}) than the original training, limiting how far they can drift from the pretrained solution. New modules (pair geometric encoder, second attention layers) are connected to the pretrained pathway through zero-initialized gates (Section 5.2.2), ensuring that at initialization the network produces exactly the same output as the pretrained model. This provides the same protection against catastrophic forgetting as parameter-efficient methods such as LoRA [?], with the added benefit that the learned gate values are directly interpretable (Section 6.6).

Table 6.1: Shared hyperparameters for all finetuning experiments. The only parameters that vary across experiments are the architectural flags (USE_PAIR_BIAS, USE_DOUBLE_ATTN, USE_GABRIEL) and the loss schedule (mat_epoch, rig_epoch).

Parameter	Value
Epochs	100
Optimizer	Adam
Base learning rate	10^{-4} ($10\times$ lower than original training)
New layer LR multiplier	$10\times$ (effective LR 10^{-3})
LR scheduler	Cosine annealing
Batch size	4 (two-piece) / 2 (multi-piece)
Points per object	5,000 (area-weighted sampling)
Min. points per fragment	30
Checkpoint selection	Best val/mat_f1
<i>Standard loss schedule (unless noted otherwise):</i>	
Segmentation loss (α)	1.0 from epoch 0
Matching loss (β)	1.0 from epoch 0
Rigidity loss (γ)	1.0 from epoch 30

Environment migration. The original Jigsaw codebase was developed for Python 3.8, PyTorch 1.10, and CUDA 11.3. We migrated the entire software stack to Python 3.10, PyTorch 2.7, and CUDA 12.8, resolving breaking changes across the PyTorch Geometric ecosystem, Open3D, GTSAM, and PyTorch Lightning (1.9→2.5). The migration also resolved out-of-memory failures that occurred with the original environment, likely due to improved memory management in newer PyTorch releases (e.g., refined CUDA caching allocator and reduced fragmentation). All experiments were conducted on a single NVIDIA RTX 4090 (24 GB). The migrated environment reproduces the original pretrained checkpoint’s evaluation metrics, confirming that the port does not affect numerical results.

New modules receive a $10\times$ higher learning rate than pretrained parameters, since they are initialized from scratch and must learn faster to catch up with the already-converged backbone. Figure 6.2 shows the resulting learning rate trajectories for all parameter groups across our experiments: the baseline uses a single parameter group with cosine annealing from 10^{-4} , while pair attention and double attention configurations use two groups: one at 10^{-4} for pretrained weights and one at 10^{-3} for new layers.

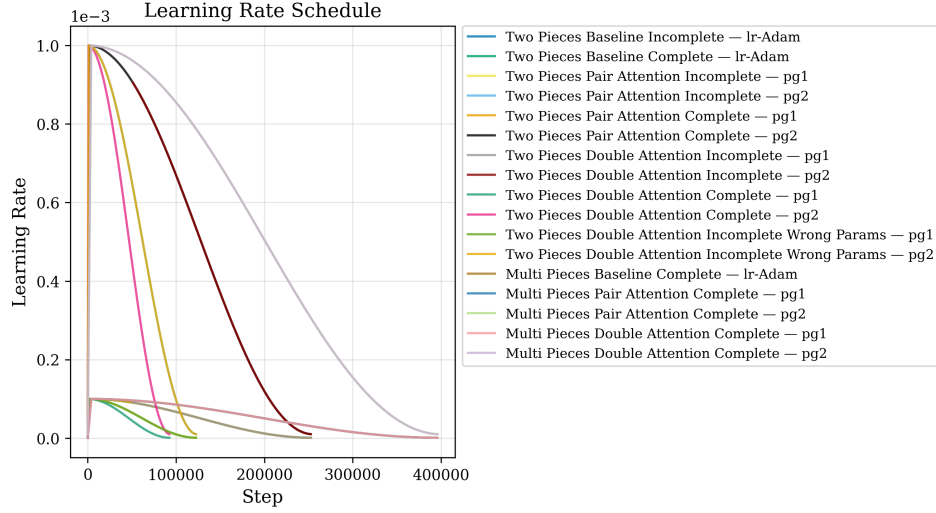


Figure 6.2: Learning rate schedules across all experiments. The baseline (single parameter group) uses cosine annealing from 10^{-4} . Pair attention and double attention models use two parameter groups: pretrained weights at 10^{-4} (pg1) and new layers at 10^{-3} (pg2). Multi-piece experiments run for more steps due to the larger training set size at the same number of epochs.

To verify that finetuning did not degrade the pretrained segmentation backbone, we monitored validation segmentation accuracy throughout training (Figure 6.3). Across all configurations, segmentation accuracy remained stable above 97%, confirming that the finetuning process did not cause catastrophic forgetting of the pre-trained features.

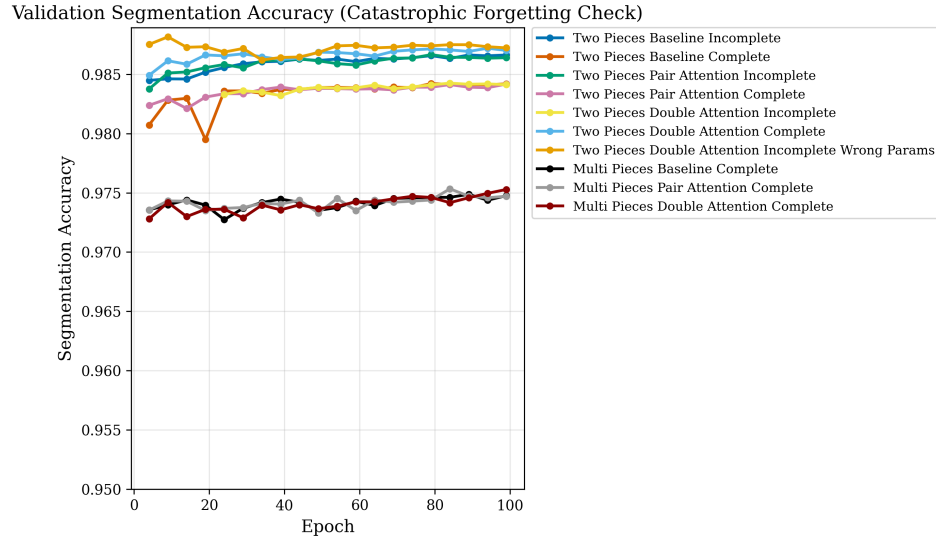


Figure 6.3: Validation segmentation accuracy across all experiments. All configurations maintain accuracy above 97% throughout training, indicating that finetuning does not degrade the pretrained segmentation backbone.

6.1.4 Scope and Limitations

It is important to note that our experiments focus on a restricted setting compared to the full Breaking Bad benchmark. Due to computational constraints, we fine-tune and evaluate on subsets of 2 pieces (pairwise assembly) and 2 to 4 pieces (limited multi-piece assembly), rather than the full 2 to 20 piece range used in the original Jigsaw evaluation. As a result, our absolute numbers are not directly comparable to published results that evaluate on the complete piece range. However, all comparisons between our proposed modifications and the baseline are conducted under identical conditions, ensuring internal validity.

6.2 Context: Published Results on Breaking Bad

Before presenting our experiments, we situate the Jigsaw baseline within the broader landscape. Table 6.2 reports published results from recent methods on the full Breaking Bad benchmark (2 to 20 pieces). These numbers serve as context but are *not* directly comparable to our results, which operate on restricted piece-count subsets.

Table 6.2: Published results on the volume-constrained Breaking Bad dataset. Everyday values for multi-piece assembly (2 to 20 pieces) are from GARF’s unified comparison [LJC⁺25]. Artifact values for Global, DGL, and PMTR are from PMTR’s pairwise evaluation [LML⁺24].

Method	Everyday				Artifact			
	$RMSE(R) \downarrow$	$RMSE(T) \downarrow$ ($\times 10^{-2}$)	$PA \uparrow$	$CD \downarrow$ ($\times 10^{-3}$)	$RMSE(R) \downarrow$	$RMSE(T) \downarrow$ ($\times 10^{-2}$)	$PA \uparrow$	$CD \downarrow$ ($\times 10^{-3}$)
Global [HZF ⁺ 20]	80.50	14.60	28.70	13.00	86.30 [†]	21.02 [†]	–	7.16 [†]
LSTM	82.70	15.10	27.50	13.30	–	–	–	–
DGL [HZF ⁺ 20]	80.30	13.90	31.60	11.80	85.46 [†]	21.65 [†]	–	6.19 [†]
Jigsaw [LSH23]	42.19	6.85	68.89	8.22	43.75	7.91	65.12	8.50
PMTR [LML ⁺ 24]	31.57	9.95	70.60	5.56	23.28 [†]	7.27 [†]	–	0.42 [†]
PHFormer [‡] [CYD24]	26.1	7.5	50.7	9.6	–	–	–	–
GARF [LJC ⁺ 25]	6.10	1.22	95.33	0.22	5.82	1.27	95.04	0.42

[†] Pairwise evaluation from [LML⁺24]. [‡] Evaluated on the vanilla (non-volume-constrained) variant.

6.3 Results on the Full Dataset

6.3.1 Two-Piece Assembly

As a first step, we fine-tuned the pretrained Jigsaw model exclusively on 2-piece fracture patterns from the full everyday training set using the standard loss schedule. Adding an ICP refinement step between RANSAC and Shonan averaging (Section 5.5.1) yielded a consistent improvement of approximately 1 percentage point on both subsets. Since ICP introduces no learnable parameters, this improvement is entirely orthogonal to the other modifications. We retain ICP in all subsequent experiments.

On the full dataset, the baseline already achieves 86.4% PA on everyday objects, leaving limited headroom for architectural improvements on in-distribution data (Table 6.3). All four modifications were evaluated: pair attention (Section 5.2.1), double attention (Section 5.2.2), and Gabriel graph filtering (Section 5.2.3) at the best retention ratio ($r=0.75$; see Section 6.7 for the tuning analysis). On everyday, the baseline remains the strongest. On the *artifact* subset, which serves as a cross-domain generalization benchmark, all modifications show improvements in pose metrics, with Gabriel producing the best chamfer distance (-10%) and double attention the best translation error.

Table 6.3: Two-piece assembly results on the **full** everyday dataset ($\sim 34,000$ training patterns). All models include ICP refinement. Results averaged over 3 runs.

Model	Everyday				Artifact			
	PA $\uparrow$	R-MAE $\downarrow$	T-MAE $\downarrow$ ($\times 10^{-2}$)	CD $\downarrow$	PA $\uparrow$	R-MAE $\downarrow$	T-MAE $\downarrow$ ($\times 10^{-2}$)	CD $\downarrow$
Baseline	86.4	16.8	2.48	2.68	72.7	26.7	6.47	11.6
+ Pair Attn	85.8	16.7	2.58	2.96	74.0	25.9	6.33	11.2
+ Double Attn	86.0	17.8	2.60	2.87	72.9	26.3	6.29	10.6
+ Gabriel	85.9	17.6	2.63	3.18	73.4	26.6	6.16	10.4

The pair attention results align with the soft regularization hypothesis (Section 5.2.1). On everyday, where the baseline already achieves well-calibrated attention, the uniform smoothing provides no benefit (-0.6 PA). On the artifact subset, however, pair attention produces the strongest improvements in part accuracy ($+1.3$) and rotation MAE (-3%), consistent with the expectation that attention smoothing helps most when the model faces unfamiliar geometry and is prone to overconfident mismatches.

6.3.2 Multi-Piece Assembly (2 to 4 Pieces)

To test whether the proposed modifications scale to higher complexity, we extended our experiments to fragments with 2 to 4 pieces (Table 6.4).

Table 6.4: Multi-piece assembly results (2 to 4 pieces) on the **complete** dataset. All models include ICP refinement. Results averaged over 3 runs.

Model	Everyday				Artifact			
	PA $\uparrow$	R-MAE $\downarrow$	T-MAE $\downarrow$	CD $\downarrow$	PA $\uparrow$	R-MAE $\downarrow$	T-MAE $\downarrow$	CD $\downarrow$
Baseline	70.5	27.2	5.17	11.0	61.8	35.1	8.23	17.2
+ Pair Attn	70.2	28.2	5.23	10.7	61.3	36.0	8.34	17.1
+ Double Attn	70.3	27.9	5.08	10.3	61.3	34.7	8.05	16.5

In the multi-piece setting, the baseline remains strong overall. Double attention continues to show the best cross-domain improvements (CD -4% , T-MAE -2%). Pair attention shows a different pattern than in the two-piece case: it improves chamfer distance on both subsets (everyday CD 10.7 vs. 11.0, artifact CD 17.1 vs. 17.2), indicating that the attention smoothing still benefits dense pose alignment, but does not improve part accuracy or angular errors.

Table 6.5: Weight evolution of the pair geometric encoder across training, measured as mean absolute difference between first and last checkpoint. The RBF centers are frozen buffers; the MLP projection layers adapt slowly, consistent with learning a low-variance bias from near-constant geometric input (Section 5.2.1).

Parameter	2-piece (red.)	2-piece (full)	Multi-piece
distance_rbf.centers	0.000 (frozen)	0.000 (frozen)	0.000 (frozen)
angle_rbf.centers	0.000 (frozen)	0.000 (frozen)	0.000 (frozen)
bias_proj layer 1	0.004 (slow)	0.008 (moderate)	0.003 (slow)
bias_proj layer 2	0.006 (slow)	0.061 (active)	0.013 (moderate)

6.4 Data Scaling: Training on a Reduced Subset

To investigate how the proposed modifications behave under data scarcity, we repeated the two-piece experiments on a reduced training set obtained by randomly dropping objects from the everyday subset, retaining approximately 63% of the original patterns ($\sim 21,500$). The artifact subset was left unchanged, providing a controlled comparison across data regimes. Table 6.6 summarizes the two configurations.

Table 6.6: Dataset configurations for the data scaling analysis. The artifact subset is identical in both settings.

Split	Reduced	Full
Everyday train	21,494 patterns	34,075 patterns
Everyday val	4,805 patterns	7,679 patterns
Artifact (unchanged)	3,651 patterns	3,651 patterns

Table 6.7 shows the results. Under data scarcity, both modifications produce clear and consistent improvements. Double attention gains +2.1 points on everyday (75.9% $\rightarrow$ 78.0%) and +4.7 points on artifact (63.8% $\rightarrow$ 68.5%), demonstrating that the second attention pass provides meaningful additional capacity when the single-pass model cannot fully capture the distribution of fracture geometries from limited training examples. Pair attention shows its strongest regularization effect: +2.2 points on artifact part accuracy (64.6% $\rightarrow$ 66.8%) and substantial improvements in all pose metrics (R-MAE -2% , T-MAE -11% , CD -13%). On everyday, the effect is neutral (-0.6 PA), further confirming the pattern that the learned attention smoothing selectively benefits out-of-distribution generalization while leaving in-distribution performance largely unaffected.

Table 6.7: Two-piece assembly on the **reduced** everyday subset ($\sim 21,500$ training patterns, $\sim 63\%$ of full). Artifact subset unchanged. All models include ICP refinement except the first baseline row. Results averaged over 3 runs.

Model	Everyday				Artifact			
	PA $\uparrow$	R-MAE $\downarrow$	T-MAE $\downarrow$ ($\times 10^{-2}$)	CD $\downarrow$	PA $\uparrow$	R-MAE $\downarrow$	T-MAE $\downarrow$ ($\times 10^{-2}$)	CD $\downarrow$
Baseline	75.9	23.5	4.83	7.26	63.8	32.6	7.18	9.82
+ ICP	76.8	22.1	4.55	6.77	64.6	31.4	7.13	9.74
+ Pair Attn	76.2	23.4	4.62	6.83	66.8	30.8	6.38	8.44
+ Double Attn	78.0	22.6	4.79	7.33	68.5	28.5	7.56	10.6

Since the artifact subset is identical across both data regimes, the artifact improvements are not an artifact of data variation; they reflect a genuine benefit of the proposed modifications when the model operates beyond its training distribution. Comparing the two regimes reveals that data quantity is the dominant factor: the full-dataset baseline (86.4%) exceeds the best reduced-subset configuration (78.0%) by over 8 points. However, when data is limited, the proposed attention extensions act as *implicit regularizers*: double attention provides additional representational capacity, while pair attention provides learned attention smoothing (Section 5.2.1).

6.5 The Effect of Loss Scheduling

During the reduced-subset experiments, one configuration was run with a non-standard loss schedule: the matching loss was delayed to epoch 9 (instead of activating immediately), and the rigidity loss was set to activate at epoch 199, effectively disabling it entirely for the 100-epoch finetuning run. This “gentle” schedule produced the best results of any configuration: 80.4% part accuracy on everyday and 69.4% on artifact, outperforming the standard-schedule double attention (78.0% and 68.5%).

The difference in training dynamics is visible in the loss curves. Under the standard schedule (Figure 6.4), all three loss terms are active early and the rigidity loss spikes at epoch 30 before decaying. Under the gentle schedule (Figure 6.5), the matching loss remains near zero until epoch 9, when it jumps sharply as it activates; the rigidity loss never appears, since its activation epoch exceeds the training duration. This delayed activation allows the segmentation and matching pathways to stabilize independently before interacting.



Figure 6.4: Training loss evolution for gated double attention on the reduced subset with the **standard** loss schedule ($\beta@0$, $\gamma@30$). All three loss components are active early; the rigidity loss produces a characteristic spike at epoch 30.

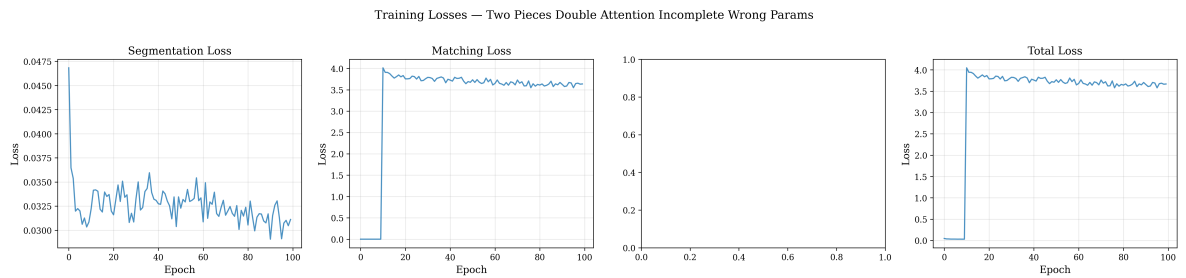


Figure 6.5: Training loss evolution for gated double attention on the reduced subset with the **gentle** loss schedule ($\beta@9$, $\gamma@never$). The matching loss activates at epoch 9 (visible as a sharp jump). The rigidity loss is absent entirely, reducing the optimization burden on the new layers.

Table 6.8: Impact of loss scheduling on gated double attention (two-piece, reduced subset). The gentle schedule delays the matching loss and disables rigidity loss entirely.

Loss Schedule	Everyday		Artifact	
	PA $\uparrow$	Rot MAE $\downarrow$	PA $\uparrow$	Rot MAE $\downarrow$
Standard ($\beta@0, \gamma@30$)	78.0	22.6	68.5	28.5
Gentle ($\beta@9, \gamma@never$)	80.4	19.6	69.4	28.7
<i>Difference</i>	+2.4	−3.0	+0.9	+0.2

This result reveals an important subtlety of finetuning with gated residual connections. Although the zero-initialized gates protect the pretrained features at initialization, the *gradients* are not gated. Under the standard schedule, the matching loss pressures the randomly initialized layers from epoch 0, creating tension with the pretrained pathway before the new layers have calibrated. The gentle schedule avoids this by letting the new layers adapt through the softer segmentation signal first, and by removing the rigidity loss entirely, reducing the optimization burden.

6.6 Gate Behavior Analysis

To understand the behavior of the gated double attention layers beyond their downstream metrics, we examined the learned gate values and the weight divergence of the second attention layers across all configurations (Table 6.9).

Table 6.9: Gate values and weight divergence of the second attention layers at the end of training. Relative divergence measures how much the second-pass layers (tf_self2, tf_cross2) have changed from their initialization (copies of the first-pass layers).

Configuration	g_s	g_c	Self2 div.	Cross2 div.	PA
2-piece, reduced (gentle)	−0.001	−0.012	0.51%	48.7%	80.4%
2-piece, full (standard)	+0.062	−0.217	0.31%	76.2%	86.0%
Multi-piece, full (standard)	+0.056	−0.184	0.43%	101.6%	70.3%

Three patterns emerge:

The self-attention gate barely opens. Across all configurations, g_s remains close to zero and the second self-attention layer diverges less than 1% from its initialization. The intra-fragment feature refinement provided by the first self-attention pass appears to be sufficient; the model finds no benefit in a second round of local reasoning.

The cross-attention gate opens significantly. The gate g_c learns to take negative values (up to -0.22), and the second cross-attention layer diverges substantially (up to 101.6%) from its initialization. This indicates that the model is actively using the second cross-attention layer, but in a *subtractive* manner: the negative gate suppresses certain inter-fragment attention patterns established by the first pass, effectively acting as a learned correction rather than an additive refinement.

Greater adaptation does not imply greater marginal gain. On the full dataset, the gates open wider and the layers diverge more than on the reduced subset, yet the marginal improvement is small. This is consistent with the data scaling finding: with sufficient data, the single-pass model already captures the patterns that the second pass provided under scarcity. The two mechanisms, additional attention capacity and additional data, address the same representational bottleneck from complementary directions.

6.7 Gabriel Graph: Retention Ratio Analysis

The Gabriel filter’s aggressiveness is controlled by the minimum keep ratio r (Section 5.2.3). We evaluated three configurations on the full everyday two-piece dataset to determine the optimal setting before reporting the test results in Table 6.3 (Figure 6.6).

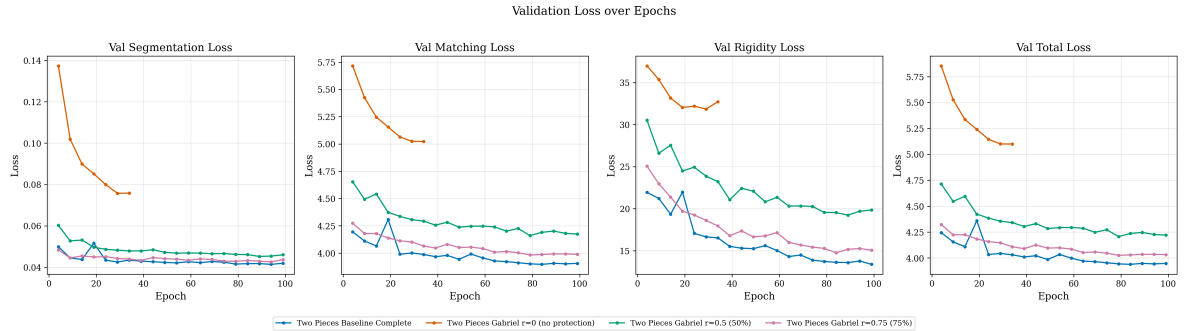


Figure 6.6: Gabriel filter retention and validation performance across configurations. All models trained for 100 epochs on the full everyday two-piece dataset.

The progression is clear: aggressive pruning ($r=0$) leaves max-pooling with almost no diversity and collapses performance. As more neighbors are preserved, performance improves monotonically. The 75% configuration achieved the highest validation mat_f1 (0.143), substantially above the baseline’s ~ 0.10 , confirming that conservative filtering preserves sufficient neighborhood diversity while removing the most topologically distant neighbors.

The validation gains did not fully transfer to test (val mat_f1 0.143 vs. test PA 85.9%, compared to baseline 86.4%). On cross-domain data (artifact), Gabriel produced the best chamfer distance (-10%) and translation error (-5%) of any modification (Table 6.3), suggesting that surface-faithful neighborhoods yield more geometrically precise features: the matches found are of higher quality, but fewer in total. The validation-to-test gap is likely caused by sensitivity to point density variations across objects: denser regions trigger more aggressive pruning, and the distribution of object densities differs between splits. Training from scratch with Gabriel-filtered neighborhoods would allow the encoder to learn features adapted to the reduced neighborhoods from the start.

6.8 Discussion

The experiments yield several conclusions. **Architectural modifications provide targeted improvements:** double attention yielded +2 to 5 points under data scarcity and consistent cross-domain gains on the artifact subset. **Data scaling has a compounding effect:** the full dataset yields over 10 points more than the reduced subset, confirming that data quantity amplifies the base architecture’s capacity; on the full dataset, the baseline already captures the patterns that the extensions provided under scarcity. **ICP refinement is reliable and orthogonal:** a consistent ~ 1 point improvement across all configurations, with no risk of overfitting. **Loss scheduling matters for gated finetuning:** a gentler activation schedule outperformed the standard one, indicating that the interaction between new gated components and pretrained representations is sensitive to optimization dynamics. **Geometric pair attention acts as a learned regularizer:** as hypothesized, Jigsaw’s preprocessing converts the pair encoder from a discriminative spatial module into a data-dependent attention smoother, consistently improving out-of-distribution generalization while remaining neutral on in-distribution performance. **Gabriel graph filtering improves cross-domain pose quality but reduces in-domain matching count:** surface-faithful neighborhoods produce higher-quality matches on out-of-distribution data (artifact), but the pruning reduces the total number of correspondences, slightly lowering the primary matching metric on in-domain data.

Figure 6.7 provides an overview of validation loss trajectories across all experiments. The most visible pattern is the separation between two-piece configurations (lower losses) and multi-piece configurations (higher losses), reflecting the increased difficulty of assembling more fragments. Within each group, the different architectural variants follow similar trajectories, consistent with the finding that the proposed modifications do not substantially alter the model’s learning dynamics when sufficient data is available.

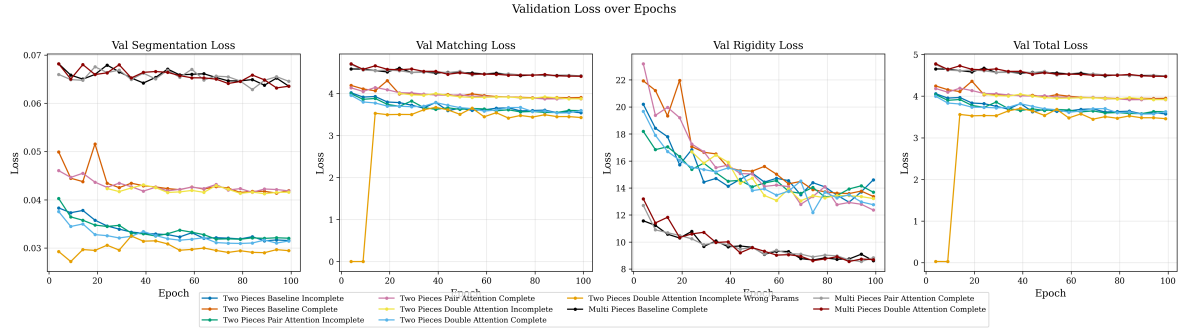


Figure 6.7: Validation loss trajectories across all experiments, broken down by loss component. Two-piece configurations (lower curves) consistently achieve lower loss than multi-piece configurations. Within each group, architectural variants (baseline, pair attention, double attention) follow similar trajectories.

Chapter 7

System Implementation and Application

7.1 System Overview

To demonstrate the models developed in this thesis and allow visual comparison of reconstruction quality across architectural variants, we built an interactive web application. The application allows users to select fractured objects from the Breaking Bad dataset, choose between eleven model variants (baseline, pair attention, double attention, and Gabriel graph filtering, each trained under up to three configurations: two-piece on the reduced subset, two-piece on the full dataset, and multi-piece on the full dataset), run inference, and inspect both the broken fragments and the assembled result in interactive 3D viewers.

7.2 Architecture

The system follows a two-server architecture with a clear separation between presentation and computation (Figure 7.1).

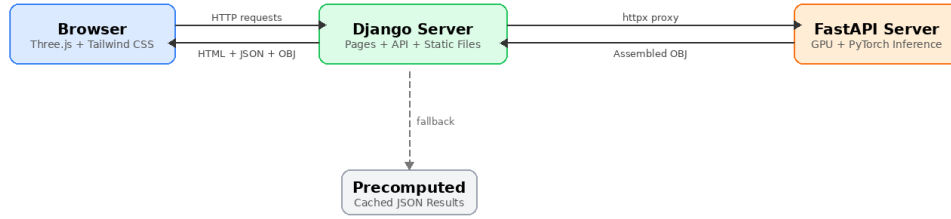


Figure 7.1: System architecture. The Django server handles user requests and serves static assets. Inference requests are proxied to the FastAPI GPU server when available, with automatic fallback to precomputed results.

The **Django web server** (Django 6.0) serves as the main entry point: it renders the user interface, hosts the 3D mesh files (Wavefront OBJ format), and exposes a REST-style JSON API for fracture metadata retrieval and inference orchestration. The **FastAPI inference server** runs on a separate GPU-equipped machine and performs the full Jigsaw inference pipeline (preprocessing, forward pass, and global alignment), returning the assembled meshes as OBJ text. Input validation rejects requests with invalid piece counts or malformed files.

To ensure the demo functions reliably during presentations without requiring dedicated hardware, the predict endpoint implements a three-tier fallback: live GPU inference when available, precomputed results from a static cache when the GPU is unreachable, and an informative error response otherwise.

Table 7.1 summarizes how the application maps to the standard software lifecycle stages.

Table 7.1: Application lifecycle summary.

Stage	Summary
Analysis	The application must allow visual comparison of fractured object reconstruction across multiple trained model variants, while remaining usable during presentations even when GPU inference is unavailable.
Design	A two-server architecture separates the web interface (Django) from GPU inference (FastAPI), with a fallback cache for offline operation.
Implementation	The frontend uses Three.js for interactive 3D visualization; the backend integrates precomputed examples and live model inference behind a unified API.
Testing	27 automated tests cover the web layer, API layer, fallback mechanism, and mocked inference pipeline (see Table 7.3).

7.3 Key Implementation Choices

Table 7.2 summarizes the technology stack.

Table 7.2: Technology stack.

Component	Technology
Web framework	Django 6.0
Inference server	FastAPI + Uvicorn
Deep learning	PyTorch + PyTorch Lightning
3D visualization	Three.js r170 (ES modules)
CSS framework	Tailwind CSS (CDN)
Point cloud processing	PyTorch Geometric, Open3D, Trimesh
Pose optimization	GTSAM (Shonan averaging)
HTTP client	httpx (Django → FastAPI proxy)

The user interface presents a single-page layout with a scrollable narrative column summarizing the thesis and a sticky interactive demo panel. The 3D viewers are built with Three.js, rendering fragment meshes with flat shading and distinct per-piece colors. Each viewer provides orbit controls for rotation, zoom, and pan, with automatic camera fitting. In the input viewer, pieces are scattered outward from their shared centroid to visually separate fragments.

7.3.1 User Workflow

Upon opening the application, the user is presented with a landing page introducing the thesis topic (Figure 7.2). Scrolling down reveals the main content area with narrative slides on the left and the interactive demo panel on the right.

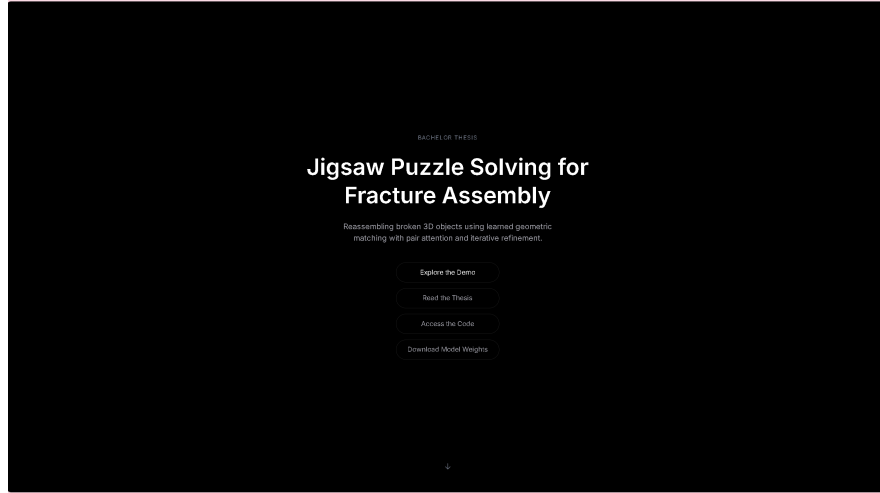


Figure 7.2: Landing page with hero section and call-to-action button.

The user selects a model variant from a dropdown (e.g., “Baseline (2 pieces)” or “Double attention (2 to 4 pieces)”), filters by object category, and browses examples using navigation controls or a fullscreen grid (Figure 7.3). The input viewer displays the fragment meshes in a scattered arrangement for inspection.

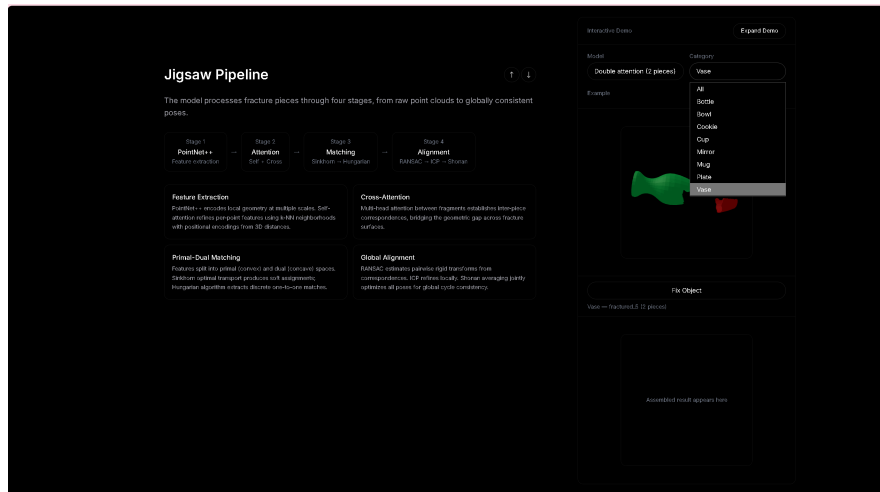


Figure 7.3: The interactive demo panel. Users select a model variant and category, browse examples, and inspect broken fragments in the 3D viewer. Clicking “Fix Object” triggers inference.

Clicking **Fix Object** triggers inference. The system either performs live GPU inference or retrieves a cached result, with a status message indicating the source.

The assembled object appears in the output viewer (Figure 7.4), where the user can rotate and zoom to inspect alignment quality at the fracture boundary (Figure 7.5).

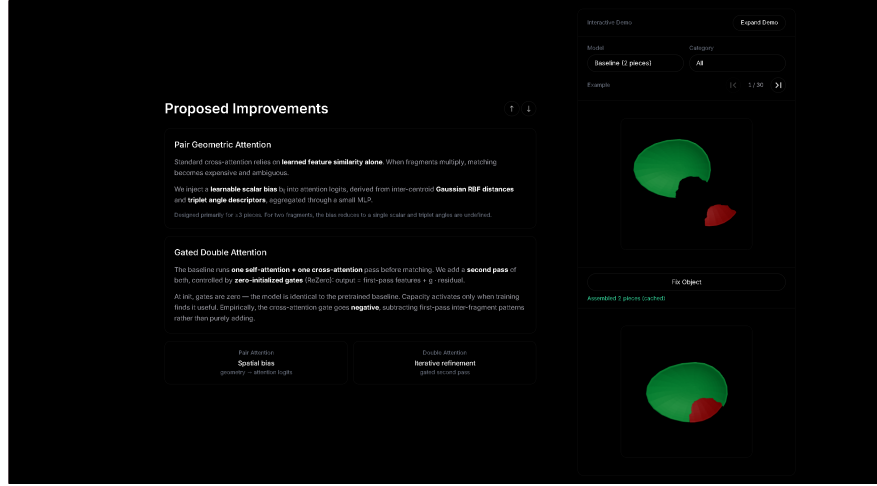
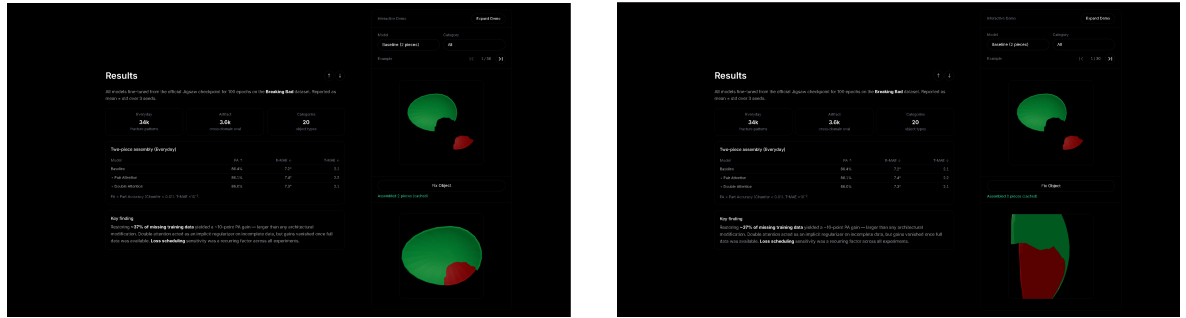


Figure 7.4: Assembled result displayed in the output viewer. The status bar indicates whether the result was computed via live GPU inference or retrieved from the precomputed cache.



(a) Zoomed view of the assembled object. (b) Close-up inspection of the fracture seam.

Figure 7.5: Detailed inspection of the reconstruction result from different viewing angles. The interactive viewer enables rotation, zoom, and pan for examining alignment quality.

An **Expand Demo** button opens a fullscreen overlay with a larger viewer and two additional tabs: **Browse Examples** (a grid of all available fractures) and **Compare Performance** (a side-by-side comparison of reconstruction quality across the eleven model variants on the same fracture input). Figure 7.6 shows the fullscreen grid view.

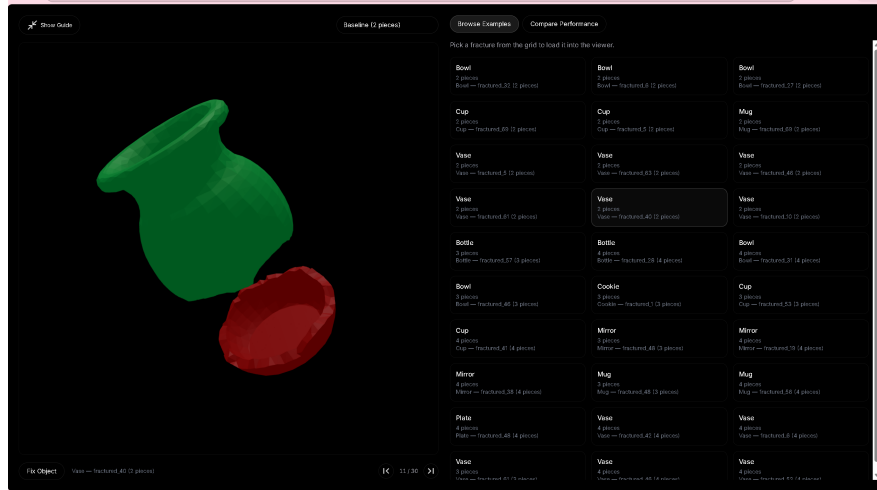


Figure 7.6: Fullscreen demo view with the Browse Examples grid for quick selection across categories.

7.3.2 Functional Requirements

The application satisfies the following functional requirements:

- **F1: Fracture visualization.** Display broken 3D fragments in an interactive viewer with rotation, zoom, and pan controls.
- **F2: Model selection.** Allow the user to choose between eleven model variants covering different architectural configurations, dataset sizes, and piece counts.
- **F3: Inference execution.** Perform end-to-end fracture assembly inference (feature extraction, segmentation, matching, and global alignment) given a set of OBJ fragments.
- **F4: Result visualization.** Display the assembled object in a separate 3D viewer for inspection.
- **F5: Graceful degradation.** Operate without a GPU server by serving precomputed results from a static cache.
- **F6: Example browsing.** Provide navigation controls and a fullscreen grid for browsing the complete set of available examples.

7.4 Software Quality Considerations

The system is designed around a separation-of-concerns principle: Django handles presentation, dataset browsing, and request orchestration, while FastAPI isolates GPU-based model inference. This separation simplifies testing and deployment,

and allows the user interface to remain functional even when live inference is unavailable.

Several validation mechanisms prevent invalid inference requests. On the Django side, user selections are restricted to known dataset examples, model variants, and piece counts. On the FastAPI side, incoming requests are validated for supported model identifiers, piece count limits, decodable OBJ content, and well-formed input data before any inference is executed.

The prediction workflow follows a proxy-and-fallback design: Django acts as a proxy between the user interface and the FastAPI inference server, first attempting live GPU inference, then falling back to precomputed cached outputs if the GPU server is unreachable, and finally returning a structured error response if neither source is available. This ensures the demo never fails silently.

Although the application is a local research demo rather than a production system, basic security measures are included: inference is restricted to predefined examples and supported model variants, request payloads are validated before processing, and browser-facing POST requests are protected through Django’s CSRF mechanism.

7.5 Testing

The application is covered by multiple automated tests across both the Django web layer and the FastAPI inference server, summarized in Table 7.3. All tests run without GPU access by mocking heavy dependencies (PyTorch, model inference), ensuring they can be executed in any development environment. The Django tests cover data models, view rendering, JSON API responses, and the GPU proxy/fallback logic. The FastAPI tests cover health checks, input validation, and end-to-end inference with mocked models.

Table 7.3: Test suite summary. All tests run without GPU by mocking PyTorch and model inference.

Component	Test Layer	Tests
Django	Data model (ID generation, display names, paths)	5
Django	View layer (page rendering, JSON API, 404 handling)	5
Django	Integration (GPU proxy, cache fallback, 503 error)	4
FastAPI	Health and model listing	7
FastAPI	Input validation (piece count, UTF-8, model variant)	5
FastAPI	End-to-end inference (mocked)	1
Total		27

Additional details on data organization, API endpoints, non-functional requirements, and deployment constraints are provided in Appendix A.

Chapter 8

Conclusions, Threats to Validity, and Future Work

8.1 Conclusions

This thesis investigated whether targeted, modular modifications to the Jigsaw framework can improve 3D fracture assembly quality, and what such experiments reveal about the interplay between architecture, data, and optimization.

The four proposed extensions (pair geometric attention, gated double attention, Gabriel graph neighborhood filtering, and ICP refinement) were evaluated systematically across multiple data regimes. The experimental evaluation yielded several findings:

- **Geometric pair attention acts as a learned soft regularizer.** Within Jigsaw’s preprocessing context, the pair encoder adapted from GPAT learns a near-uniform additive bias that smooths cross-attention distributions, functioning as a data-dependent learned temperature. This regularization consistently improves out-of-distribution generalization on the artifact subset (up to +2.2 PA and −13% CD in the two-piece reduced setting) while remaining neutral on in-distribution performance.
- **Gated double attention provides targeted improvements under data scarcity.** Double attention yielded +2 to 5 points under data scarcity and consistent cross-domain improvements on the artifact subset, demonstrating that additional attention capacity helps extract geometric patterns that a single-pass model cannot recover from limited or out-of-distribution examples.
- **Loss scheduling is critical for gated finetuning.** A gentler activation schedule outperformed the standard one, revealing that the interaction between new

gated components and pretrained representations is sensitive to when and how loss terms are introduced.

- **Gabriel graph filtering produces higher-quality matches at the cost of fewer correspondences.** Replacing kNN with Gabriel-filtered neighborhoods, a parameter-free geometric modification, improved cross-domain pose quality on artifact objects (chamfer distance -10% , translation MAE -5%), indicating that surface-faithful neighborhoods yield more geometrically precise features. However, pruning neighbors reduces the total number of correspondences found, slightly lowering the primary matching metric on in-domain data. A validation-to-test gap (val mat_f1 0.143 vs. test 0.102) suggests sensitivity to point density variations across objects.
- **ICP refinement is reliable and orthogonal.** A consistent ~ 1 point improvement across all configurations, with no risk of overfitting or interference with training dynamics.
- **Data scaling has a compounding effect.** The full dataset yields over 10 percentage points more than the reduced subset, confirming that data quantity amplifies the effectiveness of the base architecture. On the full dataset, the baseline already captures the patterns that the architectural extensions provided under scarcity, suggesting that data and architecture address the same representational bottleneck from complementary directions.

Beyond the experimental contributions, the thesis produced an environment migration from Python 3.8 / PyTorch 1.10 / CUDA 11.3 to Python 3.10 / PyTorch 2.7 / CUDA 12.8, enabling the Jigsaw codebase to run on current-generation hardware, and an interactive web application for visual comparison of assembly results across model variants. Table 8.1 provides a compact overview of the thesis scope.

Table 8.1: Thesis contribution summary.

Dimension	Description
Problem	3D fracture assembly, 6-DoF pose estimation
Baseline	Jigsaw [LSH23]
Contributions	Pair attention, double attention, Gabriel filtering, ICP, data scaling analysis
Dataset	Breaking Bad: everyday + artifact (full and reduced)
Evaluation	Two-piece, multi-piece (2 to 4), cross-domain (artifact)
Application	Django + FastAPI + Three.js interactive demo
Testing	27 automated tests (unit, integration, proxy/cache)
Environment	Migrated from PyTorch 1.10 / CUDA 11.3 to PyTorch 2.7 / CUDA 12.8

8.2 Threats to Validity

Synthetic data only. All experiments use the Breaking Bad dataset, which generates fractures through physically-based simulation. Real-world fractures involve material erosion, surface degradation, and incomplete fragments that the synthetic pipeline does not model. Results may not transfer directly to real-world applications.

Limited object categories. Training is restricted to 20 everyday object categories from PartNet. The artifact subset provides a cross-domain signal, but the overall category diversity remains narrow. Models may not generalize to domains with fundamentally different geometry, such as bones or architectural fragments.

Compute constraints. Due to the $O(N \times M)$ quadratic complexity of Jigsaw’s attention and matching stages, a single training epoch required substantial time on a single RTX 4090. This limited our study to finetuning from a pretrained checkpoint (100 epochs at 10^{-4} learning rate) rather than training from scratch (250 epochs at 10^{-3}), and restricted the piece-count range to 2 and 2 to 4 pieces instead of the full 2 to 20 range used in the original evaluation.

Statistical power. All results are averaged over 3 independent runs. While sufficient to identify large effects (e.g., the 10-point data scaling gap), smaller differences between architectural variants may not be statistically significant at this sample size.

8.3 Future Work

The threats identified above suggest several directions for future research.

Real-world evaluation. The most pressing limitation is the restriction to synthetic data. The FRACTURA dataset introduced by GARF [LJC⁺25] provides the first systematic benchmark for real-world fracture assembly, covering ceramics, bones, eggshells, and lithics. Evaluating and adapting the proposed modifications on FRACTURA, potentially using few-shot domain adaptation via LoRA as demonstrated by GARF, would test whether the findings of this thesis hold beyond the synthetic setting.

Category generalization. To address the limited category diversity, a cross-domain finetuning strategy similar to GARF’s approach could be employed: training on the full everyday set and then finetuning with a small number of objects from

the target domain (e.g., artifact or FRACTURA categories). This would test whether the implicit regularization effect observed for double attention under data scarcity also manifests under domain shift.

Efficient attention architectures. The compute constraints that limited this thesis to finetuning stem directly from the quadratic complexity of Jigsaw’s attention mechanism. Two complementary solutions exist in the literature: FlashAttention [?], which reduces the memory footprint of standard attention through tiling and recomputation without changing the output, and the proxy-based matching paradigm of PMTR [LML⁺24], which achieves sub-quadratic complexity by projecting fragment features onto a shared learned proxy space. Integrating either approach would enable training from scratch on the full piece-count range within practical hardware budgets.

Backbone upgrade to Point Transformer V3. All experiments in this thesis use the PointNet++ backbone inherited from Jigsaw. GARF [LJC⁺25] demonstrates that replacing PointNet++ with Point Transformer V3 (PTv3), a substantially more expressive feature extractor with built-in positional encodings and surface normal awareness, yields large improvements in assembly quality. Retraining the Jigsaw pipeline with a PTv3 backbone would test whether the proposed modifications (particularly Gabriel filtering and pair attention) remain beneficial when the base features are already richer, or whether they become redundant when paired with a stronger encoder.

Gabriel filtering from pretraining. Our experiments showed that Gabriel graph neighborhood filtering improves cross-domain pose quality but slightly reduces the number of correspondences found. The validation-to-test gap suggests sensitivity to point density variations across objects. Training PointNet++ with Gabriel-filtered neighborhoods from scratch, rather than applying the filter during finetuning, would allow the encoder to learn features adapted to surface-faithful connectivity from the start, potentially closing this gap. A learnable keep ratio, rather than the fixed hyperparameter r used in this work, could further improve adaptability.

Discriminative pair attention via iterative pose refinement. In Jigsaw’s current preprocessing, the pair encoder operates as a soft regularizer because the geometric descriptors have low variance. Extending Jigsaw with an iterative refinement loop, where pose estimates from each forward pass are fed back to update the geometric descriptors for the next iteration, would provide the pair encoder with progressively

more informative spatial relationships, similar to the geometric recycling mechanism used in GPAT [LCML23]. This would test whether the encoder can transition from its regularization role to a discriminative spatial bias, potentially combining both benefits.

Bibliography

- [BMM⁺] Thomas Bachlechner, Bodhisattwa Prasad Majumder, Huanru Henry Mao, Garrison W. Cottrell, and Julian McAuley. Rezero is all you need: Fast convergence at large depth. Preprint. Under review.
- [CLT⁺22] Yun-Chun Chen, Haoda Li, Dylan Turpin, Alec Jacobson, and Animesh Garg. Neural shape mating: Self-supervised object assembly with adversarial shape priors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12724–12733, 2022.
- [Cut13] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems*, volume 26, 2013.
- [CYD24] Wenting Cui, Runzhao Yao, and Shaoyi Du. Phformer: Multi-fragment assembly using proxy-level hybrid transformer. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence*, pages 1408–1416. AAAI Press, 2024.
- [DRW⁺20] Frank Dellaert, David M Rosen, Jing Wu, Robert Mahony, and Luca Carlone. Shonan rotation averaging: Global optimality by surfing $so(p)^n$. In *European Conference on Computer Vision (ECCV)*, pages 292–308. Springer, 2020.
- [FB81] Martin A Fischler and Robert C Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.
- [FSTF⁺11] Thomas Funkhouser, Hijung Shin, Corey Toler-Franklin, Antonio García Castañeda, Benedict Brown, David Dobkin, Szymon Rusinkiewicz, and Tim Weyrich. Learning how to match fresco fragments. *Journal on Computing and Cultural Heritage (JOCCH)*, 4(2):1–13, 2011.
- [GS69] K. Ruben Gabriel and Robert R. Sokal. A new statistical approach to geographic variation analysis. *Systematic Zoology*, 18(3):259–278, 1969.

- [GVL13] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.
- [HFG⁺06] Qi-Xing Huang, Simon Flöry, Natasha Gelfand, Michael Hofer, and Helmut Pottmann. Reassembling fractured objects by geometric matching. *ACM Transactions on Graphics (TOG)*, 25(3):569–578, 2006.
- [HNR22] Abhinav Narayan Harish, Rajendra Nagar, and Shanmuganathan Raman. Rgl-net: A recurrent graph learning framework for progressive part assembly. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 647–656, 2022.
- [HZF⁺20] Jialei Huang, Guanqi Zhan, Qingnan Fan, Kaichun Mo, Lin Shao, Baoquan Chen, Leonidas Guibas, and Hao Dong. Generative 3d part assembly via dynamic graph learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 6315–6326, 2020.
- [JHC⁺21] Benjamin Jones, Dalton Hildreth, Duowen Chen, Ilya Baran, Vladimir G. Kim, and Adriana Schulz. Automate: A dataset and learning approach for automatic mating of cad assemblies. *ACM Transactions on Graphics*, 40(6):1–18, 2021.
- [JJZH24] Hanwen Jiang, Zhenyu Jiang, Yue Zhao, and Qixing Huang. Leap: Liberate sparse-view 3d modeling from camera poses. In *The Twelfth International Conference on Learning Representations*, 2024. Department of Computer Science, The University of Texas at Austin.
- [LCML23] Jiahao Li, Chaoran Cheng, Jianzhu Ma, and Ge Liu. Geometric point attention transformer for 3d shape reassembly, 2023. arXiv preprint.
- [LHH24] J. Lu, G. Hua, and Q. Huang. Jigsaw++: Imagining complete shape priors for object reassembly. *Journal Name*, 2024.
- [LJC⁺25] Sihang Li, Zeyu Jiang, Grace Chen, Chenyang Xu, Siqi Tan, Xue Wang, Irving Fang, Kristof Zyskowski, Shannon P. McPherron, Radu Iovita, Chen Feng, and Jing Zhang. Garf: Learning generalizable 3d reassembly for real-world fractures. *arXiv preprint arXiv:2504.05400*, 2025.
- [LK10] John P. Laughlin and Robert L. Kelly. Experimental analysis of the practical limits of lithic refitting. *Journal of Archaeological Science*, 37(2):427–433, 2010.
- [LML⁺24] Nahyuk Lee, Juhong Min, Junha Lee, Seungwook Kim, Kanghee Lee, Jaesik Park, and Minsu Cho. 3d geometric shape assembly via efficient

- point cloud matching. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of PMLR, 2024.
- [LMS⁺20] Yichen Li, Kaichun Mo, Lin Shao, Minhyuk Sung, and Leonidas Guibas. Learning 3d part assembly from a single image. In *European Conference on Computer Vision (ECCV)*, pages 664–682. Springer, 2020.
- [LSH23] Jiaxin Lu, Yifan Sun, and Qixing Huang. Jigsaw: learning to assemble multiple fractured objects. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, Red Hook, NY, USA, 2023. Curran Associates Inc.
- [MZC⁺19] Kaichun Mo, Shilin Zhu, Angel X. Chang, Li Yi, Subarna Tripathi, Leonidas J. Guibas, and Hao Su. Partnet: A large-scale benchmark for fine-grained and hierarchical part-level 3d object understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 909–918, 2019.
- [QSMG17] Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 652–660, 2017.
- [QYSG17] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems*, volume 30, pages 5099–5108, 2017.
- [RFB15] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pages 234–241. Springer International Publishing, 2015.
- [S⁺22] S. Sellán et al. Breaking bad: A dataset for geometric fracture and re-assembly. In *NeurIPS Datasets and Benchmarks*, 2022.
- [TFBW⁺10] Corey Toler-Franklin, Benedict Brown, Tim Weyrich, Thomas Funkhouser, and Szymon Rusinkiewicz. Multi-feature matching of fresco fragments. *ACM Transactions on Graphics (TOG)*, 29(6):1–12, 2010.
- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.

- [WZX⁺20] Rundi Wu, Yixin Zhuang, Kai Xu, Hao Zhang, and Baoquan Chen. Pq-net: A generative part seq2seq network for 3d shapes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 829–838, 2020.
- [YWLM11] Zhao Yin, Li Wei, Xin Li, and Mary Manhein. An automatic assembly and completion framework for fragmented skulls. In *2011 International Conference on Computer Vision*, pages 2532–2539. IEEE, 2011.
- [ZJ16] Qingnan Zhou and Alec Jacobson. Thingi10k: A dataset of 10,000 3d-printing models. *arXiv preprint arXiv:1605.04797*, 2016.
- [ZPK18] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. Open3d: A modern library for 3d data processing. *arXiv preprint arXiv:1801.09847*, 2018.

Appendix A

Application Details

A.1 Data Organization

Fracture examples are stored as static OBJ files organized in a hierarchical directory structure:

```
meshes/{two_pieces,multi_pieces}/  
    <category>/<mesh_hash>/fractured_0/  
        piece_0.obj, piece_1.obj, ...
```

At startup, the Django application automatically discovers all valid fractures by walking this directory tree. Each fracture is assigned a deterministic 12-character ID derived from its path, ensuring stable URLs across restarts. The discovery logic filters for directories starting with `fractured_` and containing at least two `piece_*.obj` files.

A.2 API Endpoints

The Django server exposes three endpoints:

- `GET /`: Renders the main page with fracture lists injected into the template context.
- `GET /api/fracture/<id>/`: Returns JSON metadata and static OBJ URLs for a specific fracture.
- `POST /api/predict/`: Accepts a fracture ID and model variant, proxies to the GPU server or returns cached results.

The FastAPI inference server exposes:

- `GET /health`: Returns server status and CUDA availability.
- `GET /models`: Lists all eleven available model variants with descriptions and maximum piece counts.
- `POST /predict`: Accepts OBJ files as multipart uploads, runs the full Jigsaw inference pipeline (preprocessing, forward pass, global alignment), and returns the assembled meshes as OBJ text.

The server validates all inputs: it rejects requests with fewer than 2 pieces, more pieces than the selected model supports (2 for two-piece models, 4 for multi-piece), and files that are not valid UTF-8 text.

A.3 Non-Functional Requirements

- **Performance:** Live inference completes within 120 seconds (the proxy timeout). Precomputed fallback responses are served in under 100ms.
- **Portability:** The Django server runs on any Linux environment with Python 3.12+. The inference server requires CUDA-capable hardware with at least 8 GB VRAM.
- **Maintainability:** The codebase is modular: data processing, model components, training infrastructure, web application, and inference server are clearly separated. Configuration is externalized through environment variables and YAML files.
- **Security:** CSRF protection is enabled on all POST endpoints. Session cookies are configured with `HttpOnly` and `SameSite=Lax` flags. The inference server restricts CORS origins via environment configuration.